

“Hadoop Architecture and Map Reduce Programming Framework”

What is Big Data

- Wikipedia big data
 - An all-encompassing term for any collection of data sets so **large** and **complex** that it becomes **difficult** to process using on-hand data management tools or traditional data processing applications.

Who is generating Big Data?

Social



User Tracking & Engagement



Homeland Security



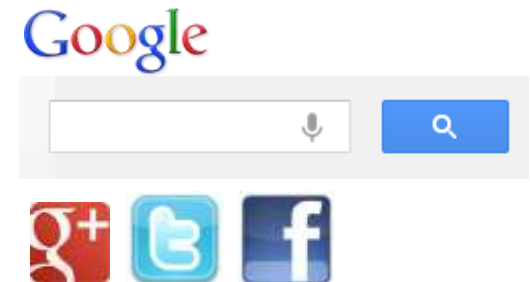
eCommerce



Financial Services

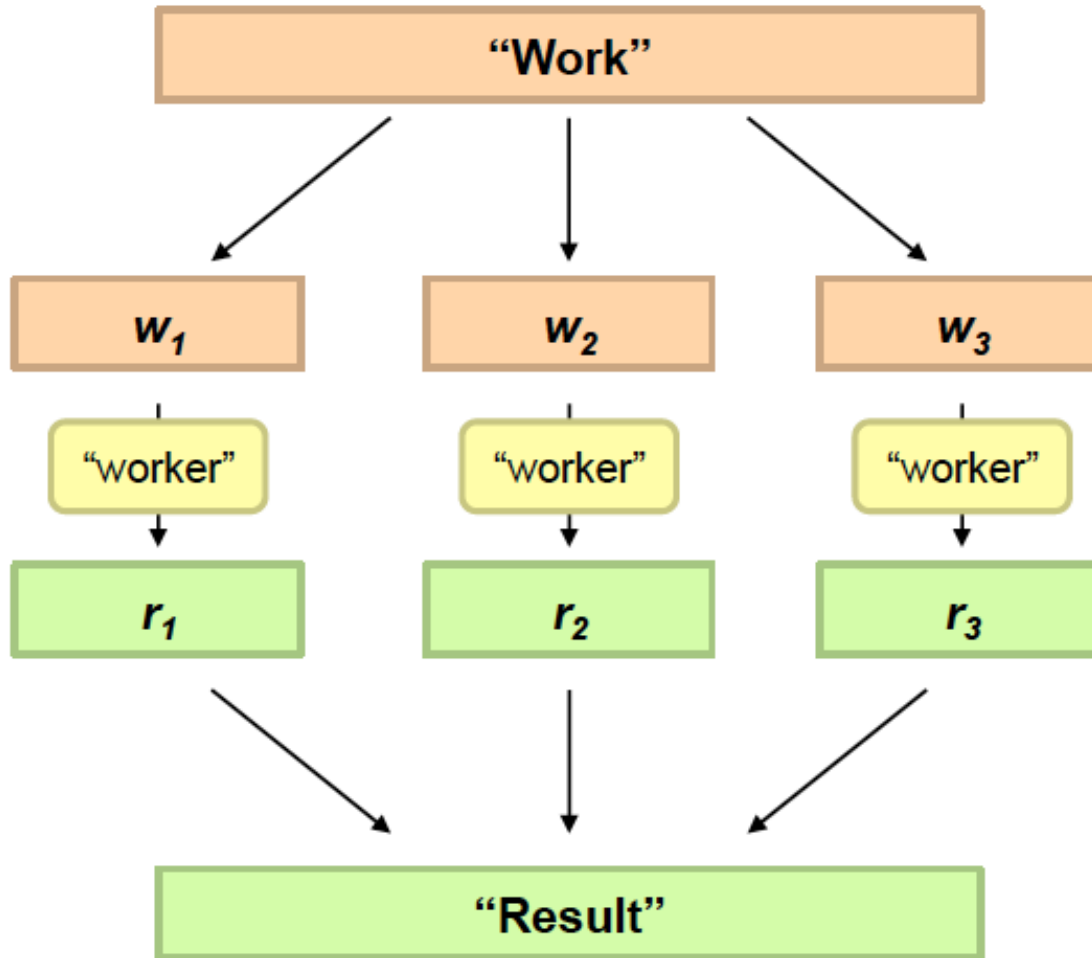


Real Time Search



- Philosophy to Scale for Big Data?

Divide and Conquer



Divide Work



Combine Results

Distributed processing is non-trivial

- How to assign tasks to different workers in an efficient way?
- What happens if tasks fail?
- How do workers exchange results?
- How to synchronize distributed tasks allocated to different workers?



Big data storage is challenging

- Data Volumes are massive
- Reliability of Storing PBs of data is challenging
- All kinds of failures: Disk/Hardware/Network Failures
- Probability of failures simply increase with the number of machines ...



One popular solution: Hadoop



Hadoop Cluster at Yahoo! (Credit: Yahoo)

HDFS Architecture: Master-Slave

Master



Name Node (NN)

Secondary Name Node
(SNN)

Data Node (DN)



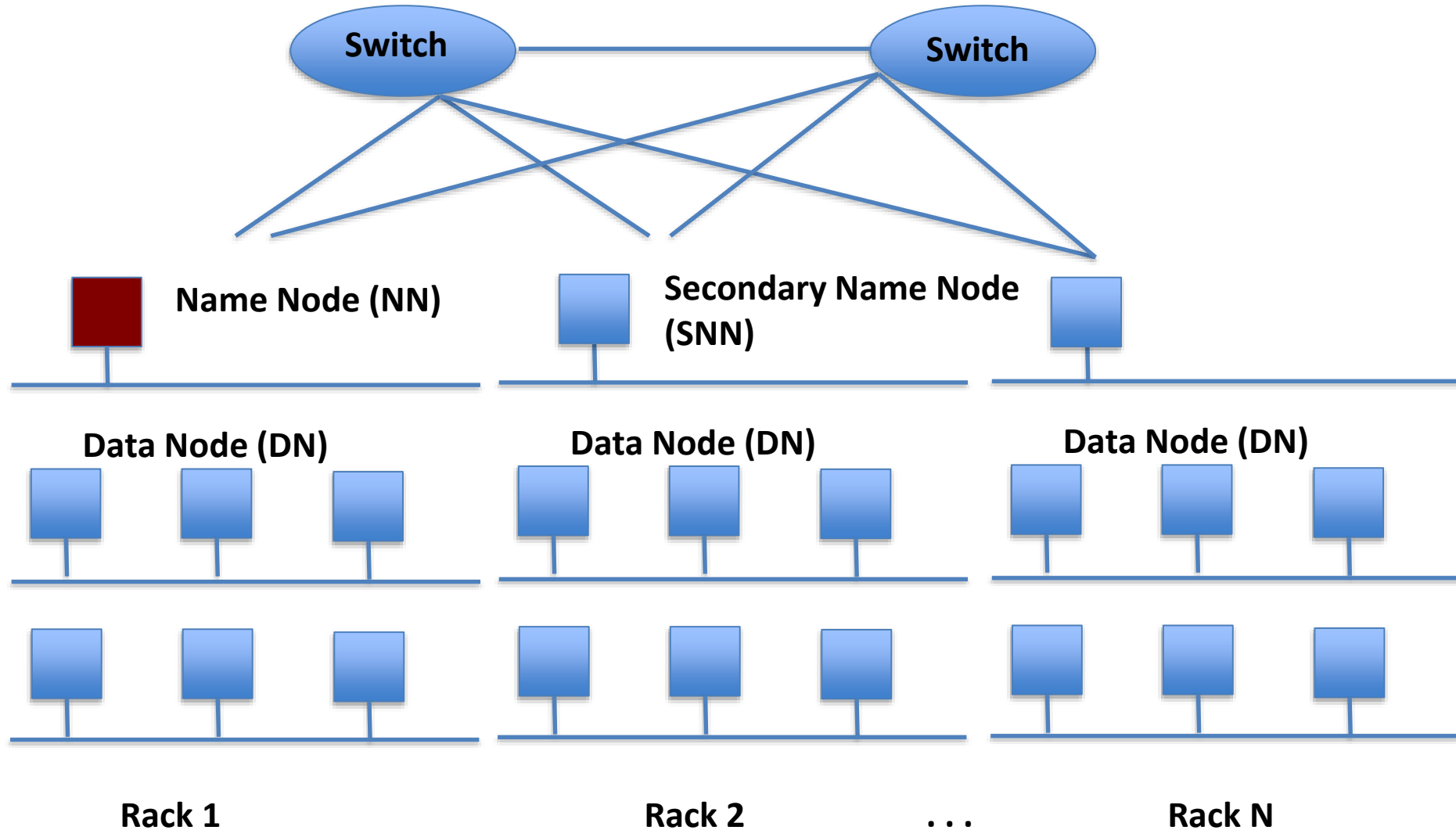
Slaves

Single Rack Cluster

- Name Node: Controller
 - File System Name Space Management
 - Block Mappings
- Data Node: Work Horses
 - Block Operations
 - Replication
- Secondary Name Node:
 - Checkpoint node

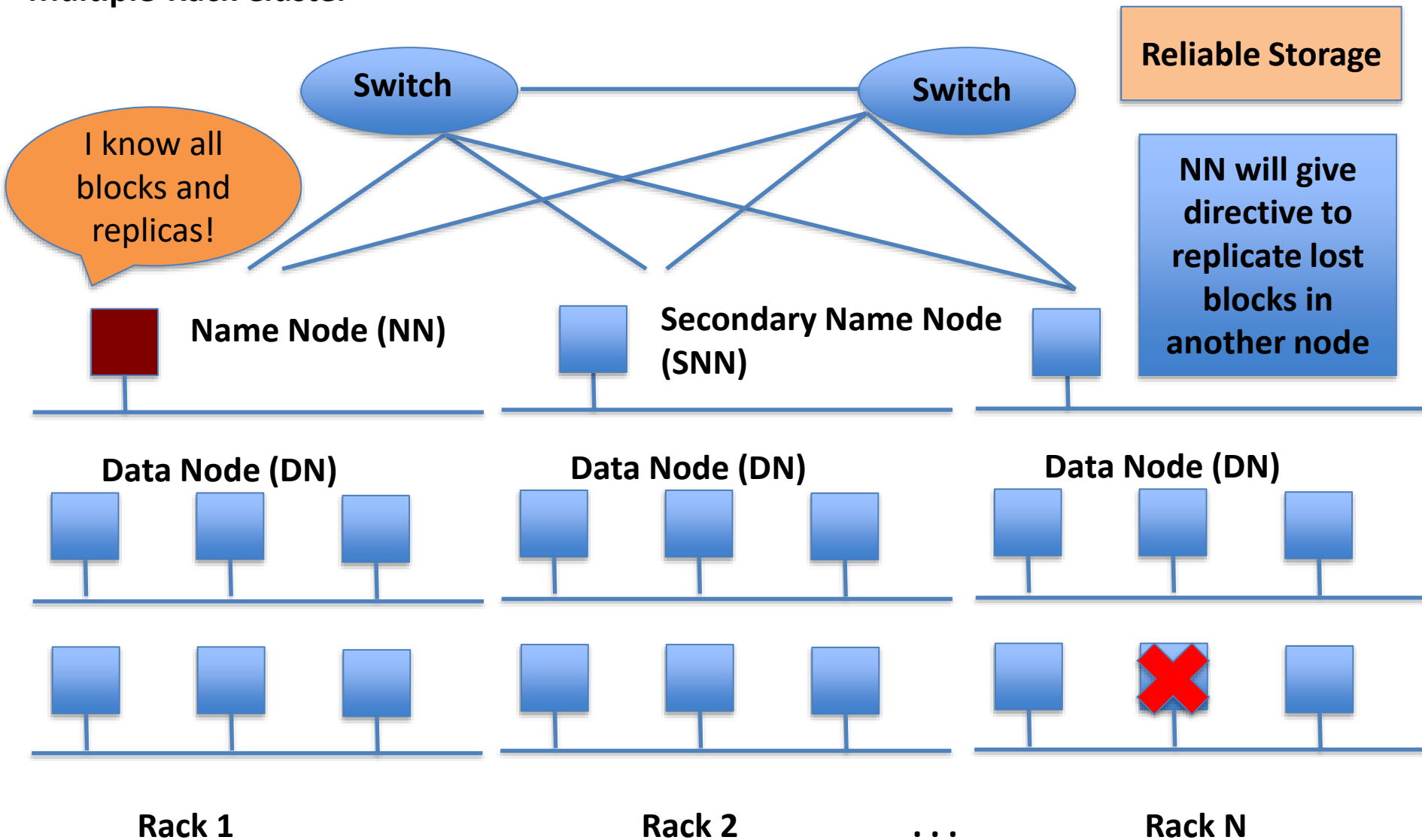
HDFS Architecture: Master-Slave

Multiple-Rack Cluster



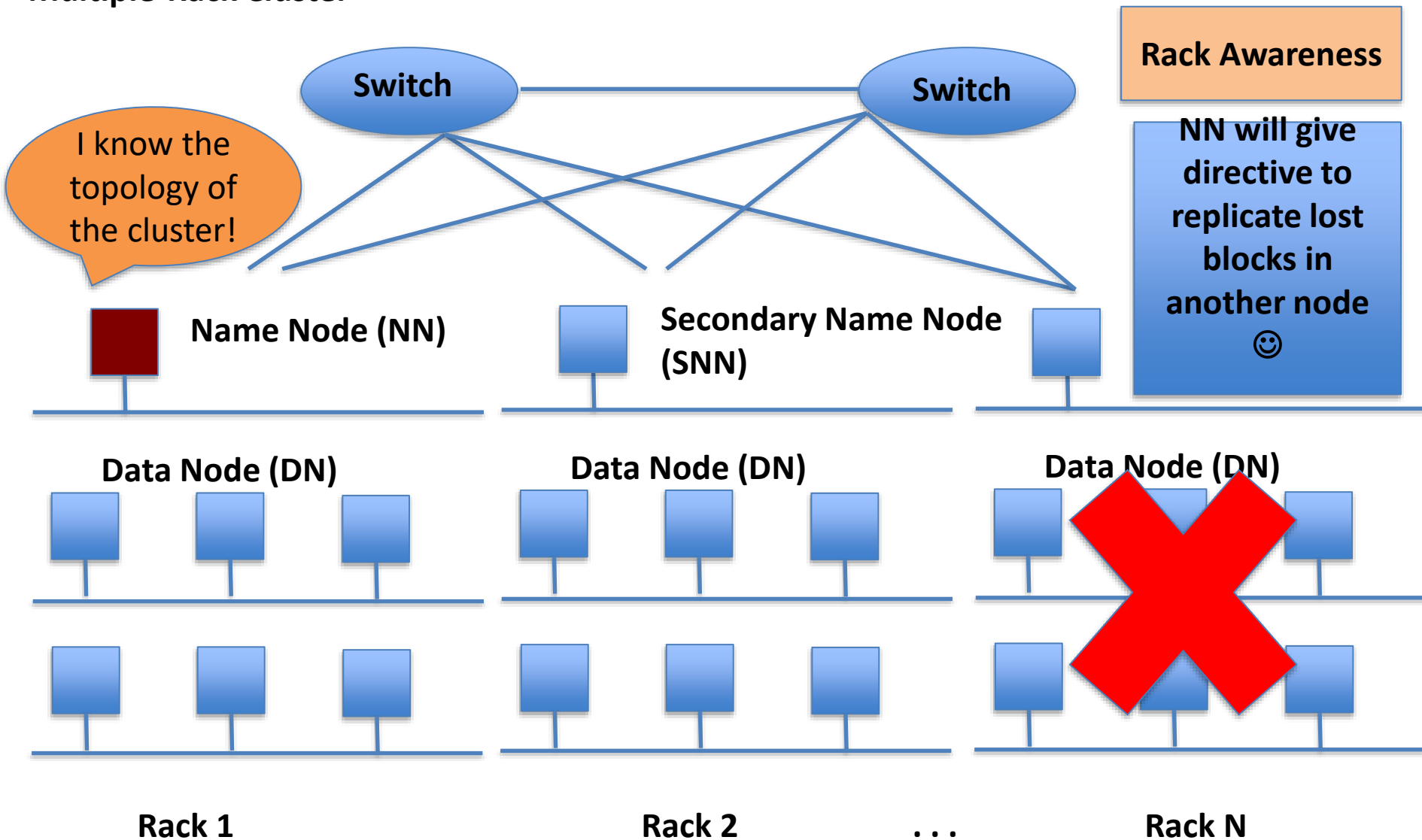
HDFS Architecture: Master-Slave

Multiple-Rack Cluster



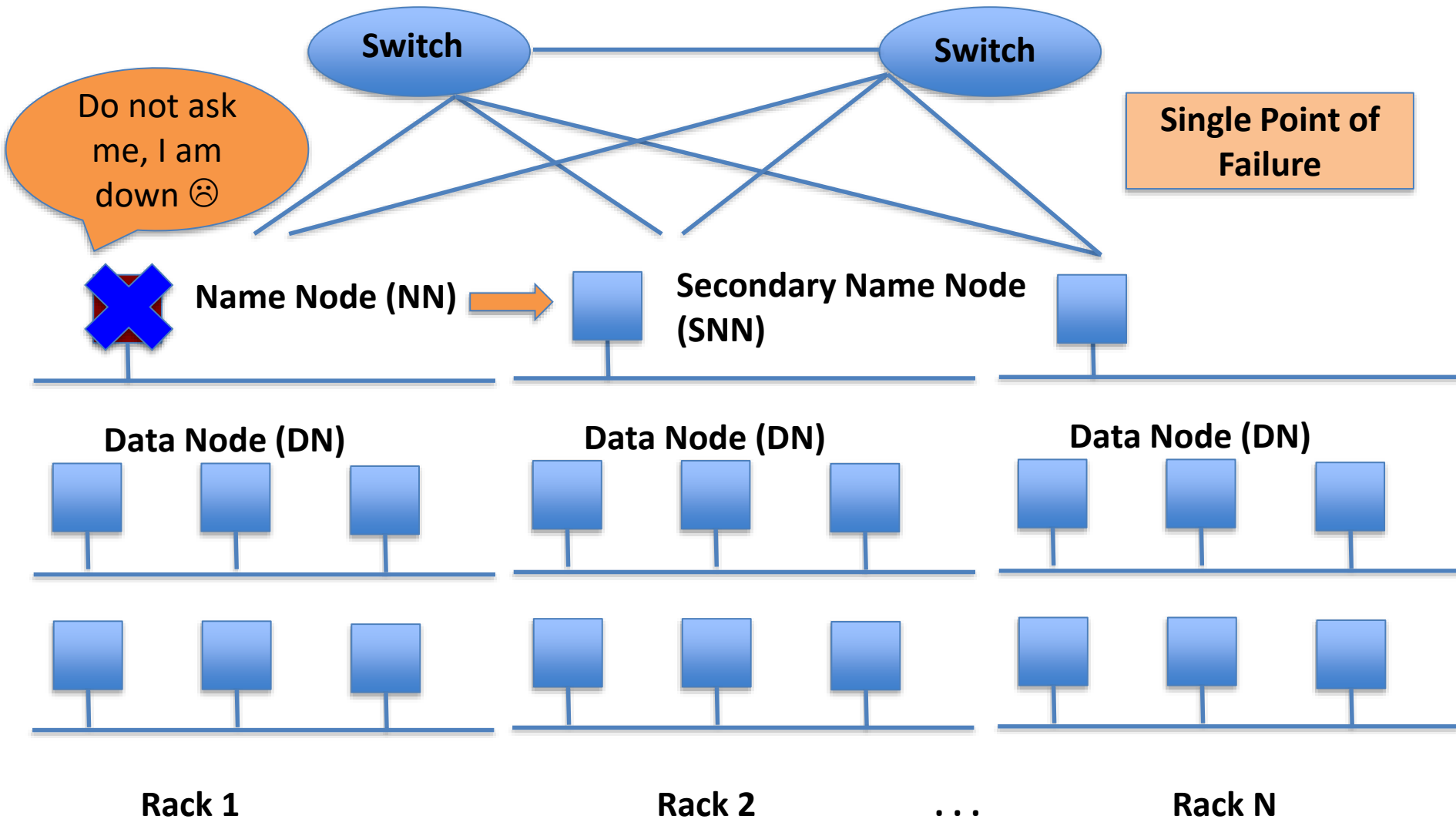
HDFS Architecture: Master-Slave

Multiple-Rack Cluster



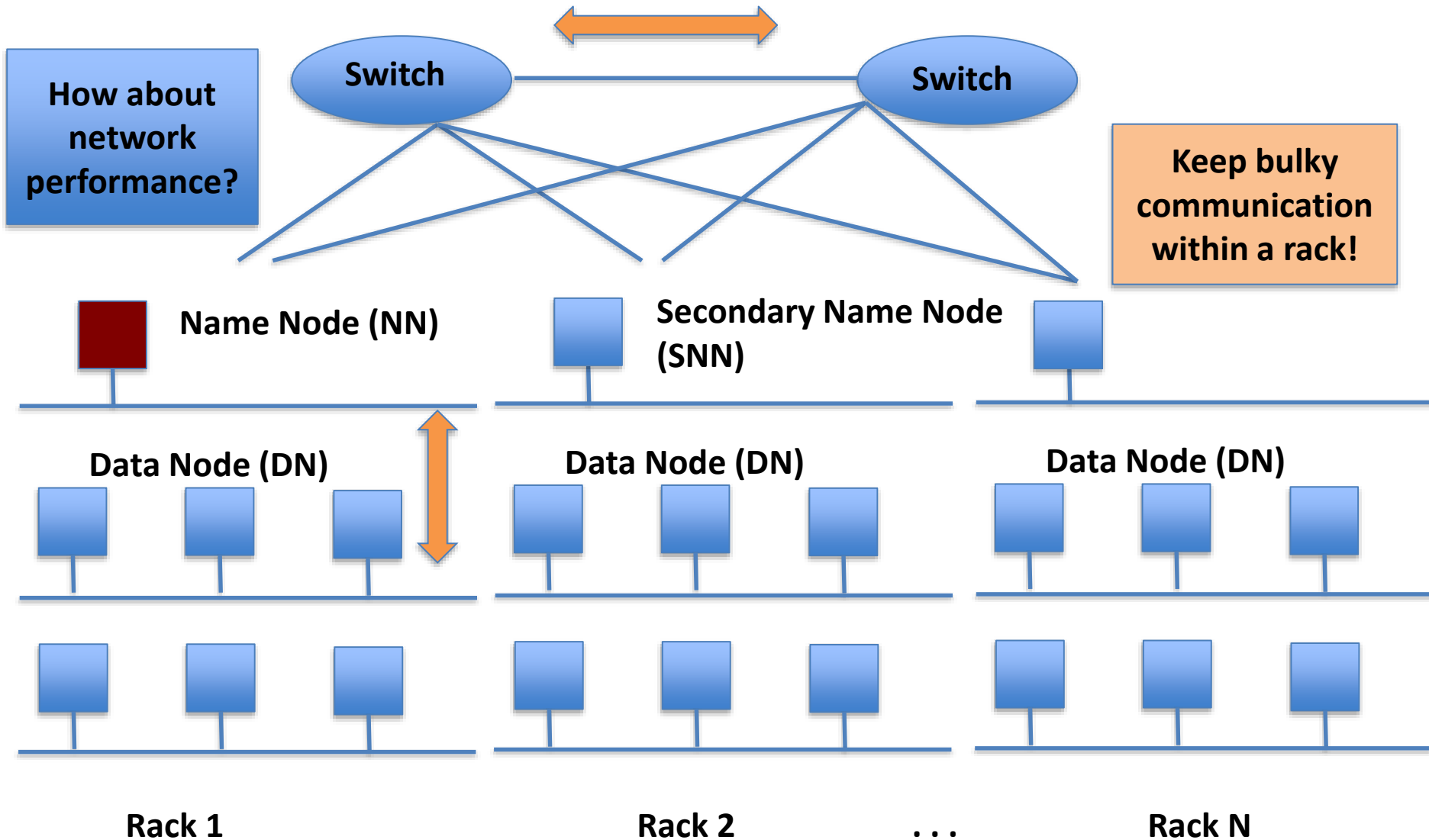
HDFS Architecture: Master-Slave

Multiple-Rack Cluster

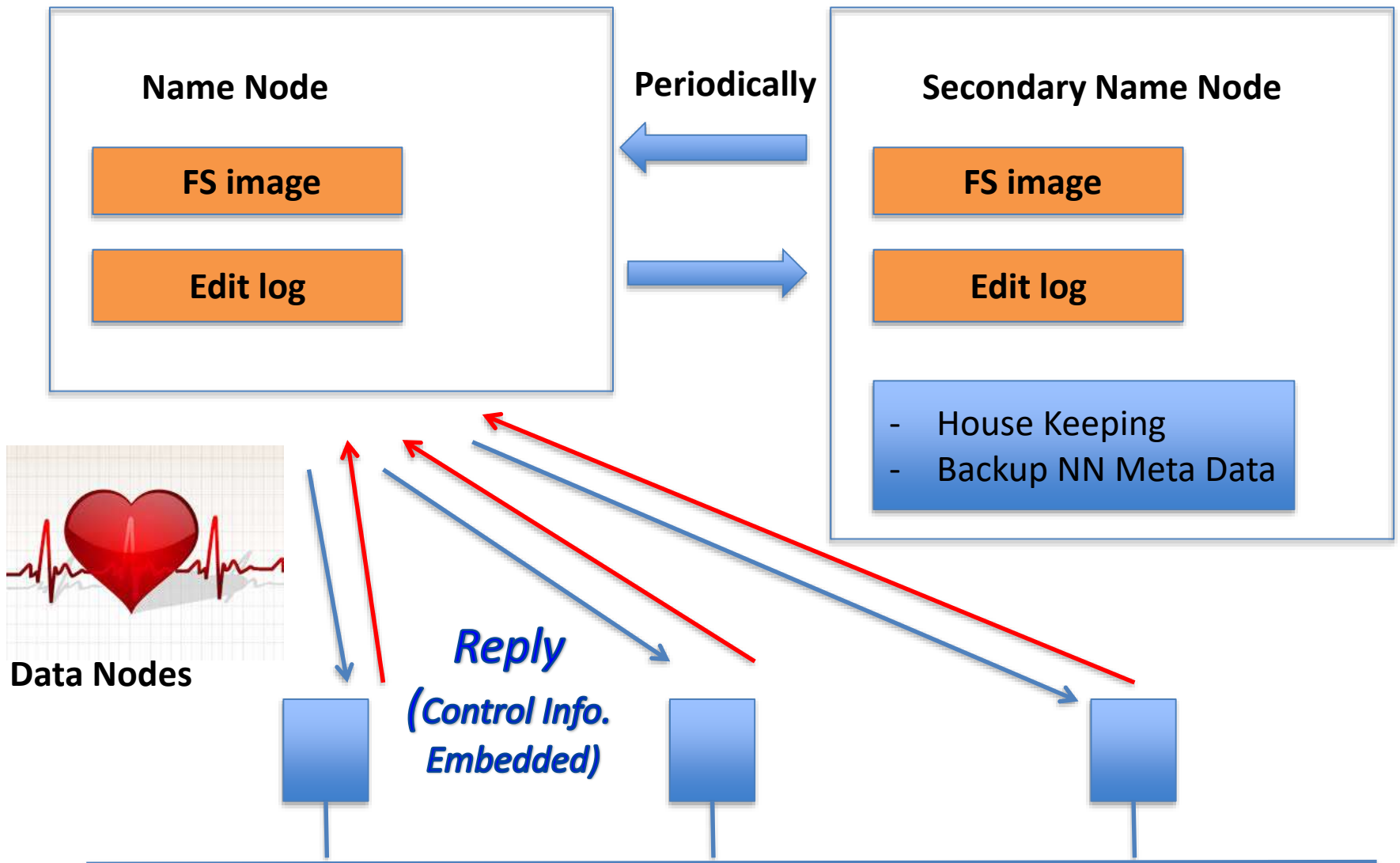


HDFS Architecture: Master-Slave

Multiple-Rack Cluster



HDFS Inside: Name Node



HDFS Inside: Blocks

- Q: Why do we need the abstraction “Blocks” in addition to “Files”?
- Reasons:
 - File can be larger than a single disk
 - Block is of fixed size, easy to manage and manipulate
 - Easy to replicate and do more fine grained load balancing

- Robustness

Possible Failures

- The primary objective of HDFS is to store data reliably in the presence of failures
- Three common failures that it must handle are:
 - DataNode failure
 - NameNode failure
 - Network partition

DataNode Failure and Heartbeat

- A crashed DataNode or a network partition can cause a subset of DataNodes to lose connectivity with the NameNode
- NameNode detects this by the absence of a heartbeat
 - NameNode marks these DataNodes, and does not send requests to them
 - Data registered to the failed DataNode is not available to the HDFS
 - Death of a DataNode may cause some blocks to require more replication

Re-Replication

- Sometimes Blocks in the system may fall below the required replication factor
- This can occur for a number of reasons
 - A DataNode has become unavailable
 - A replica may become corrupted
 - A hard disk on a DataNode may fail
 - The replication factor may have been increased

Namespace

- A hierarchical tree structure, similar to a traditional file system. It consists of directories, subdirectories, and files.
- The metadata associated with files and directories, such as file names, permissions, and block locations, is stored in the namespace. This metadata is managed and maintained by the NameNode.

HDFS Client

- HDFS client refers to a software component or library that enables applications and users to interact with the HDFS file system.
- The HDFS client provides a programming interface, command-line tools, and libraries that allow users and applications to perform operations on files and directories stored in HDFS.

Data Integrity

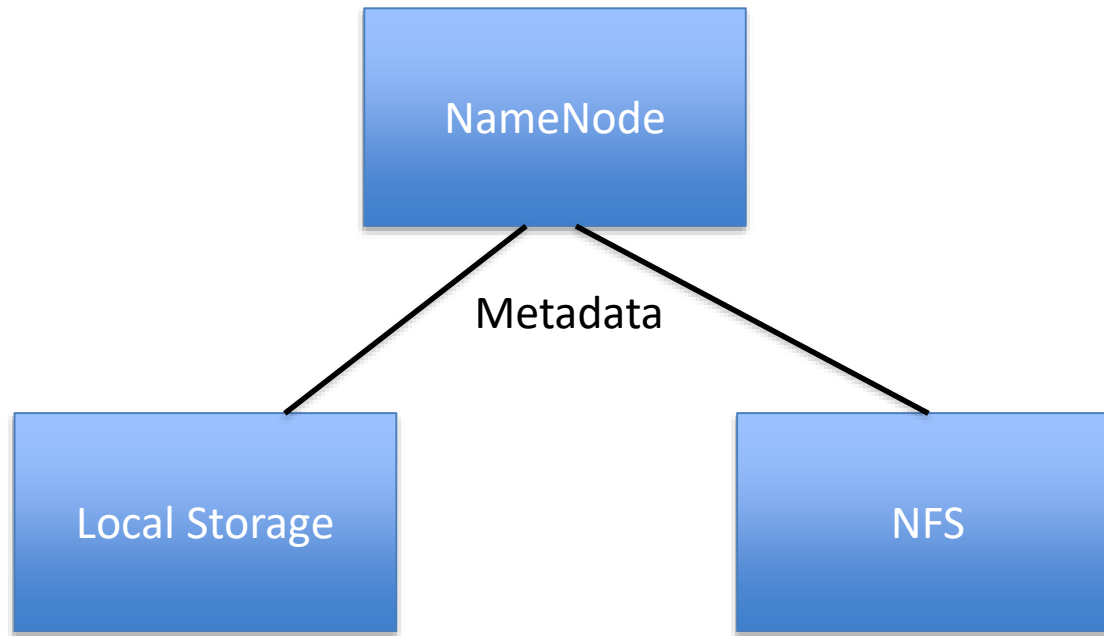
- What if a block of data fetched from a DataNode arrives corrupted
 - Fault in a storage device
 - Network faults
 - Buggy software
- An HDFS client creates a checksum for every block of its file and stores it in the HDFS namespace
- When the client retrieves the contents of a file it verifies that they match...if not it must retrieve the block from another replica

MetaData Disk Failure

- **FsImage** and **EditLog** are central data structures of HDFS Corruption of these files can cause an entire HDFS instance to become non-functional.
- A NameNode can be configured to maintain multiple copies of these files. These copies are updated synchronously
- MetaData is not data intensive
- The NameNode is a potential single point of failure. This currently requires manual intervention

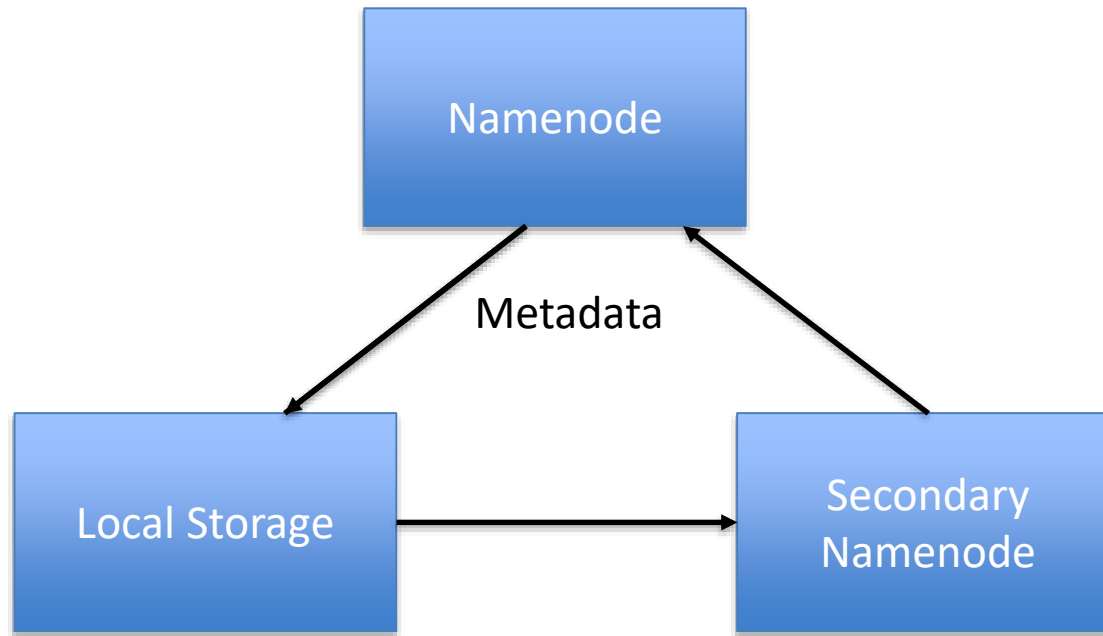
Name Node-Single Point of Failure

1. Keeping a backup of the name node image & Editlogs on two locations



Name Node-Single Point of Failure

2. Keeping a secondary Name node



Name Node-Single Point of Failure

3. Keeping a standby namenode

Cluster Rebalancing

- HDFS architecture is compatible with data rebalancing schemes
- A scheme may automatically move data from one DataNode to another if the free space on a DataNode is falling below a certain threshold
- A scheme may dynamically create and place additional replicas and rebalance other data if there is sudden high demand for a particular file
- These types of rebalancing are not yet implemented

- **Data Organization**

HDFS-Data Block

- HDFS support write-once-read-many with reads at streaming speeds.
- A typical block size is 64MB (or even 128 MB).
- A file is chopped into 64MB chunks and stored.
- Advantages
 - helps fitting big files into small discs
 - leaves less unused space on the disc
 - Optimizes the transfer
 - distributes the load to multiple machines

HDFS

- Read
- Write

Distributed File System

- A distributed file system is a type of file system that allows multiple users to access and manage the same data stored on different nodes or computers in a network.
- In a distributed file system, the data is stored across multiple nodes in a way that makes it appear as if it is stored on a single machine, while providing the ability to scale out storage and processing capabilities as needed.

- Data replication: The ability to replicate data across multiple nodes to ensure data availability and reliability.
- Data distribution: The ability to divide the data into smaller chunks and distribute them across multiple nodes for improved scalability and performance.
- Data access: The ability for multiple users to access and manipulate the same data from different nodes in a network.
- Data management: The ability to manage the data and metadata, such as permissions, ownership, and data placement, in a coordinated and consistent manner.

Interfaces in distributed file system?

- In a distributed file system, interfaces define the communication and interaction protocols between the various components or layers of the system.
- These interfaces help standardize the way different components communicate and ensure interoperability.

- **File System API**
- **Network File System (NFS) Protocol**
- **Common Internet File System (CIFS) Interface**
- **Distributed File System Protocol (DFS)**

- **Object Storage Interface**
 - **Amazon S3 Interface**
 - **Google Cloud Storage Interface**

Data flow in distributed file system

- Write Request
- Data Placement
- Data Transmission

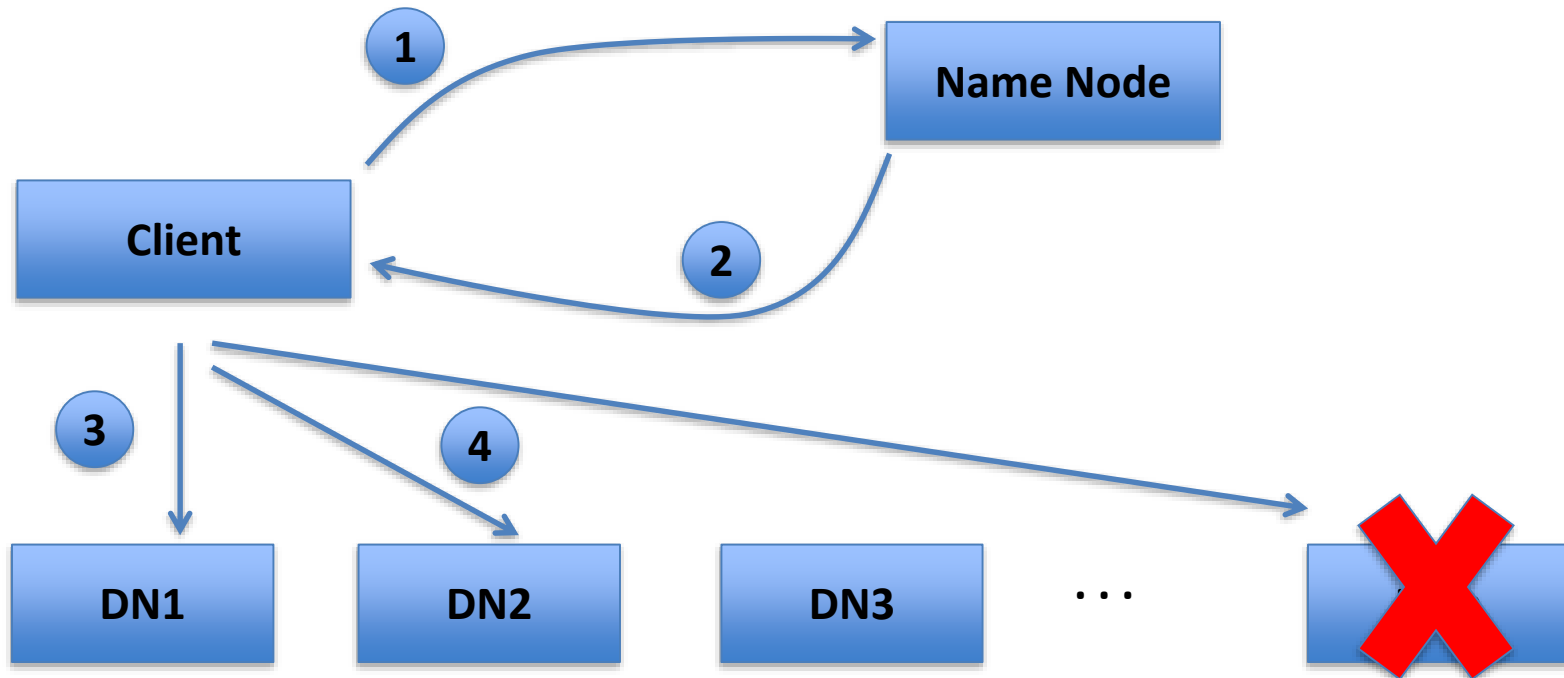
Data flow in distributed file system

- **Data Storage**
- **Read Request**

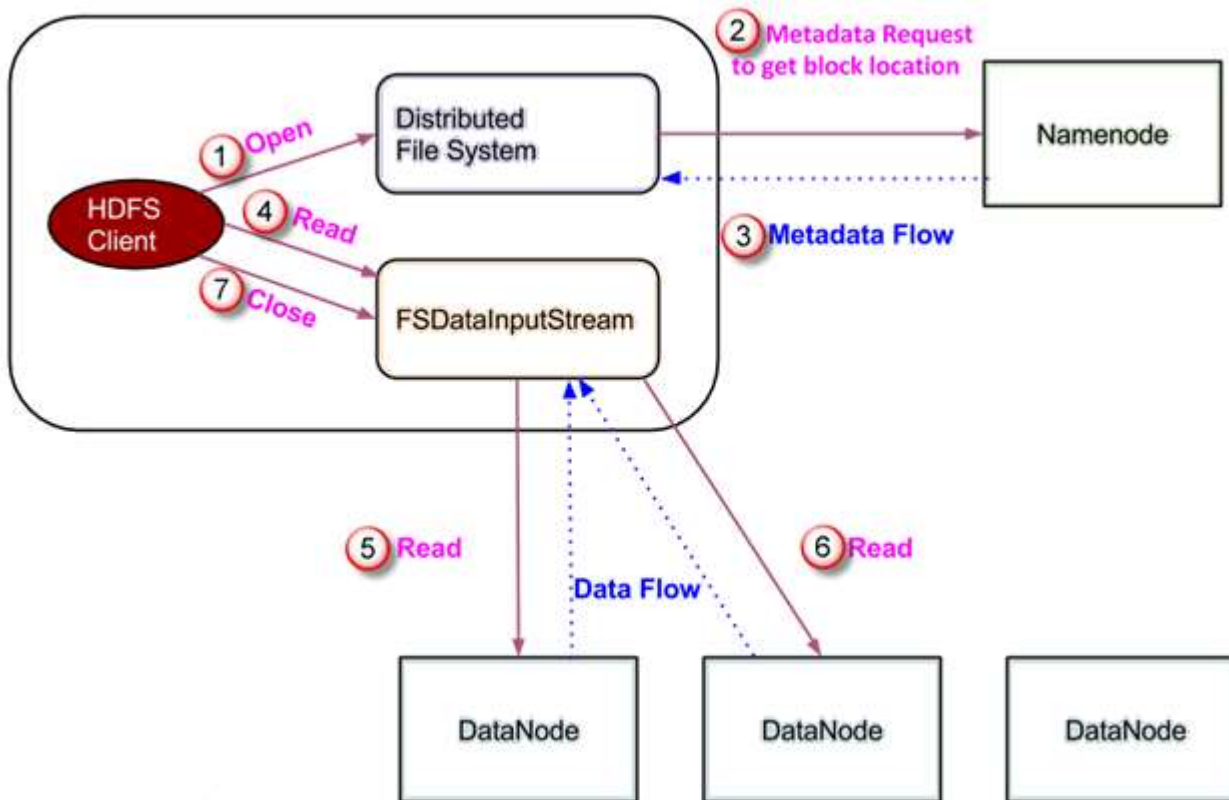
Data flow in distributed file system

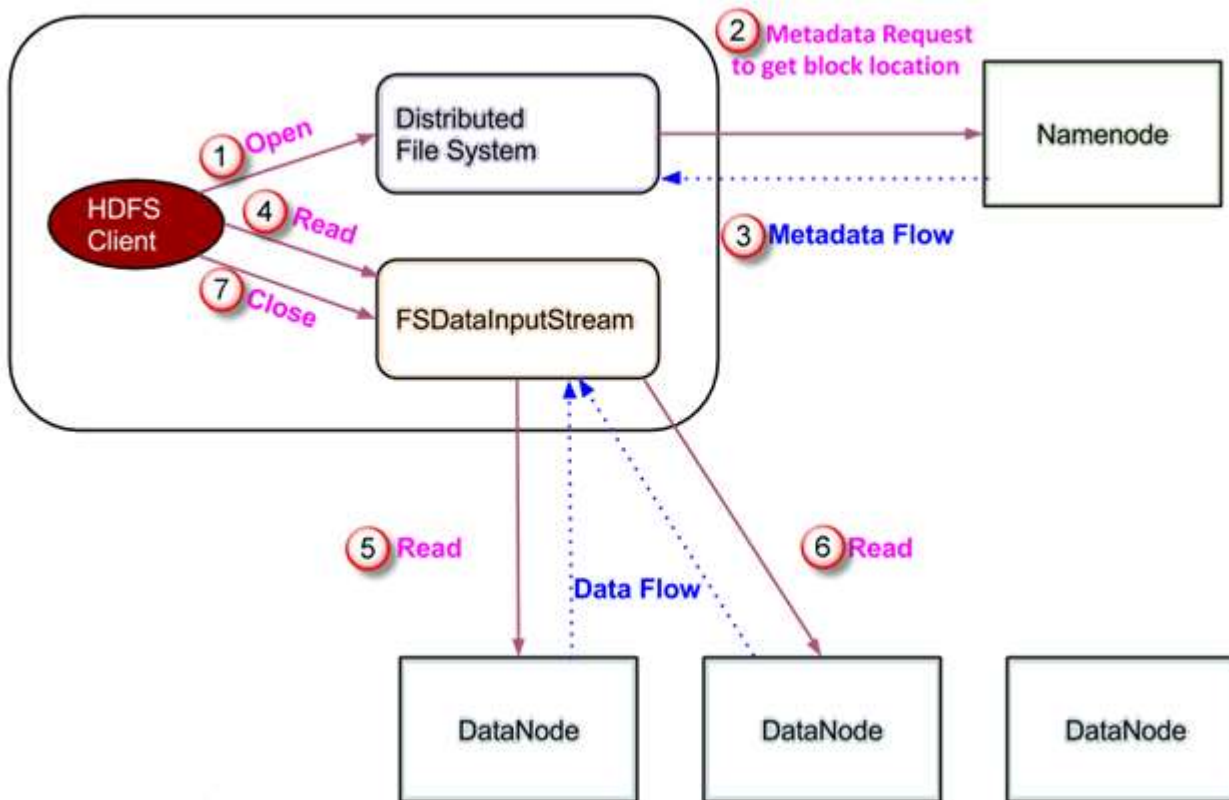
- **Data Retrieval**
- **Data Reconstruction**

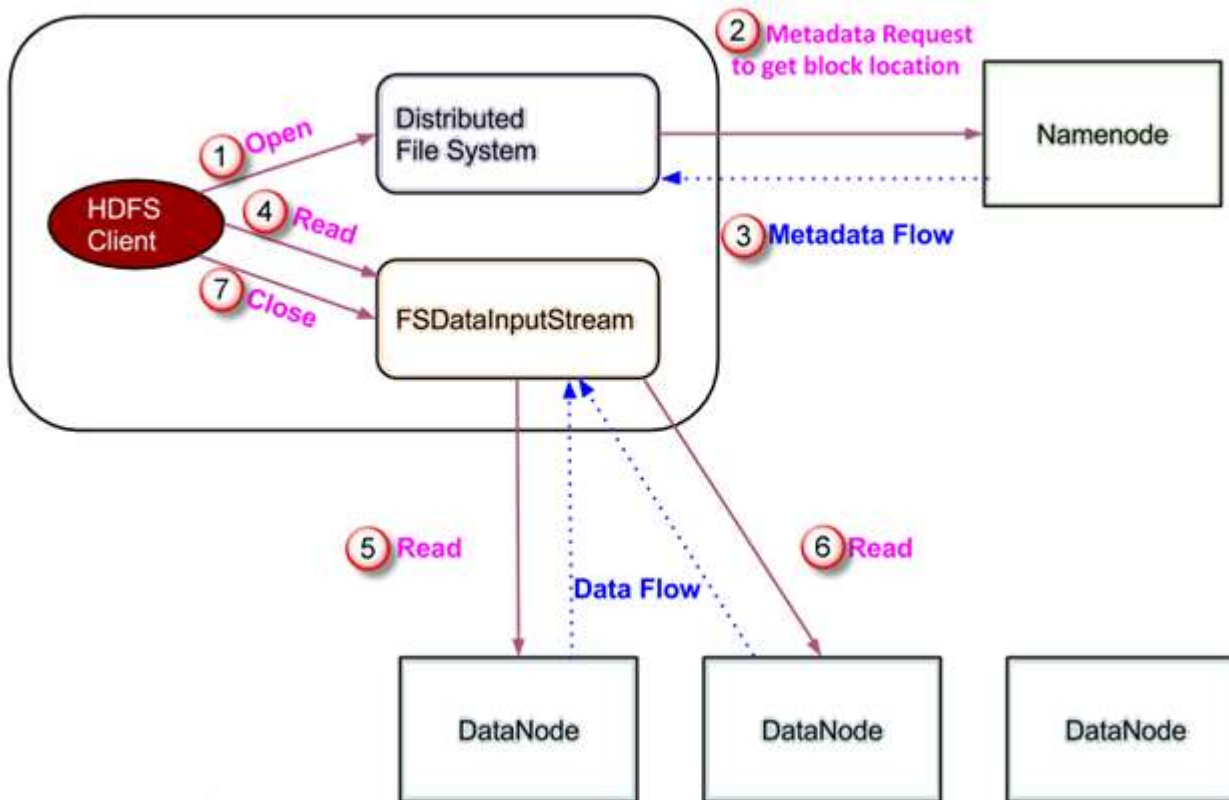
HDFS Inside: Read



1. Client connects to NN to read data
2. NN tells client where to find the data blocks
3. Client reads blocks directly from data nodes (without going through NN)
4. In case of node failures, client connects to another node that serves the missing block







HDFS Inside: Read

- Q: Why does HDFS choose such a design for read? Why not ask client to read blocks through NN?
- Reasons:
 - Prevent NN from being the bottleneck of the cluster
 - Allow HDFS to scale to large number of concurrent clients
 - Spread the data traffic across the cluster

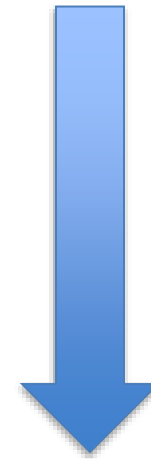
HDFS Inside: Read

- Q: Given multiple replicas of the same block, how does NN decide which replica the client should read?
- HDFS Solution:
 - Rack awareness based on network topology

HDFS Inside: Network Topology

- The critical resource in HDFS is **bandwidth**, distance is defined based on that.
- Measuring bandwidths between any pair of nodes is too complex and **does not scale**.
- **Basic Idea:**

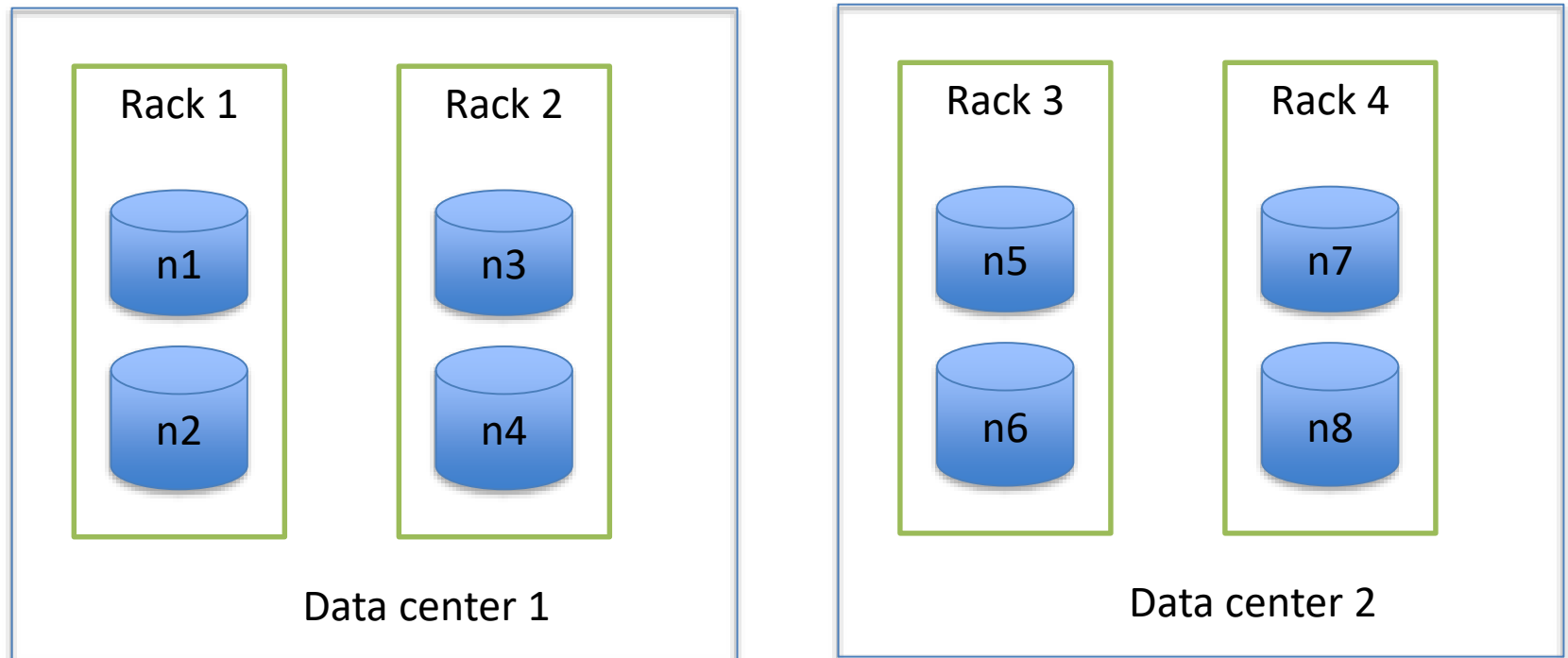
- Blocks on the same node
- Different nodes on the same rack
- Nodes on different racks in the same data center (cluster)
- Nodes in different data centers



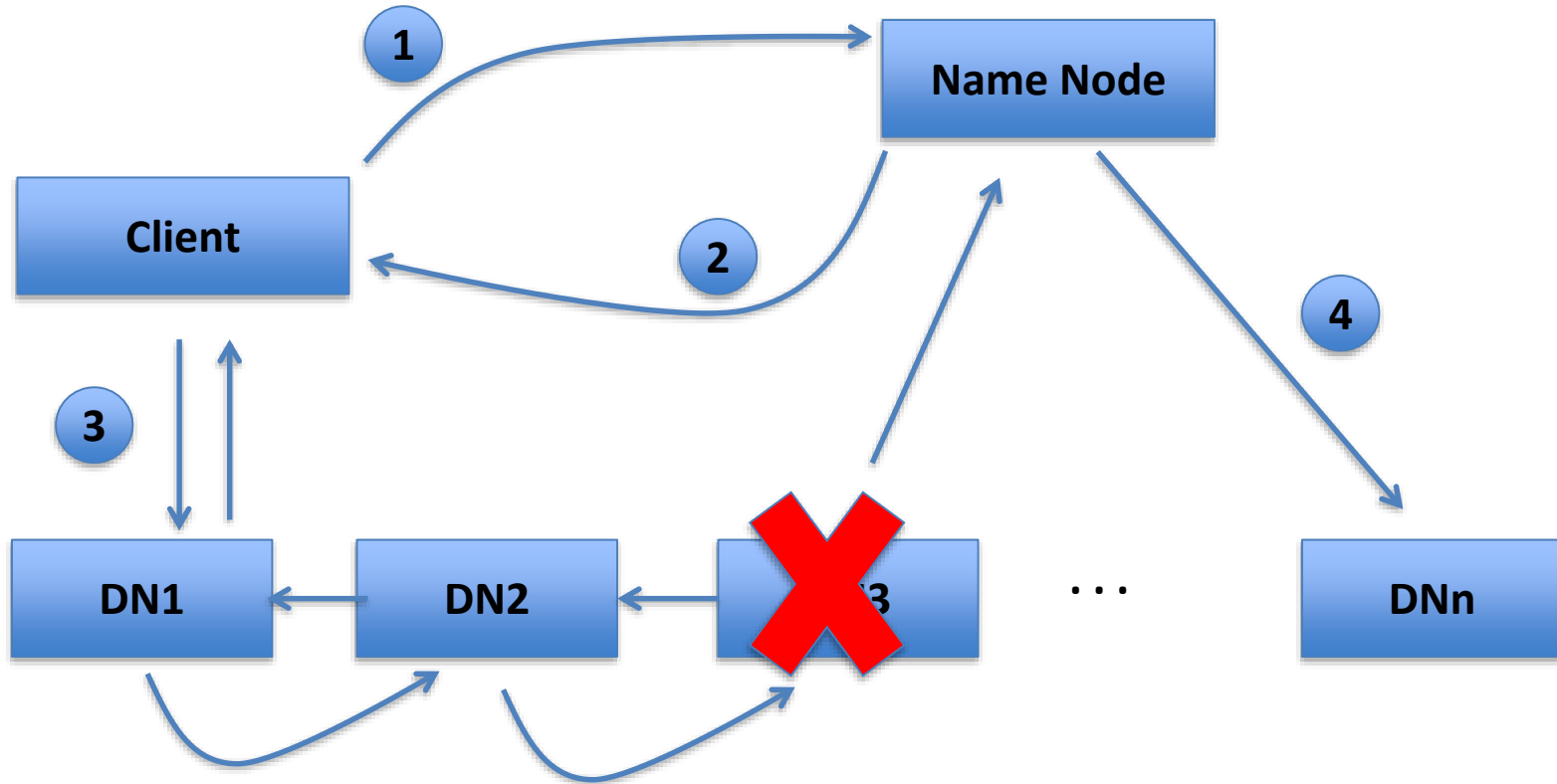
**Bandwidth
becomes less**

HDFS Inside: Network Topology

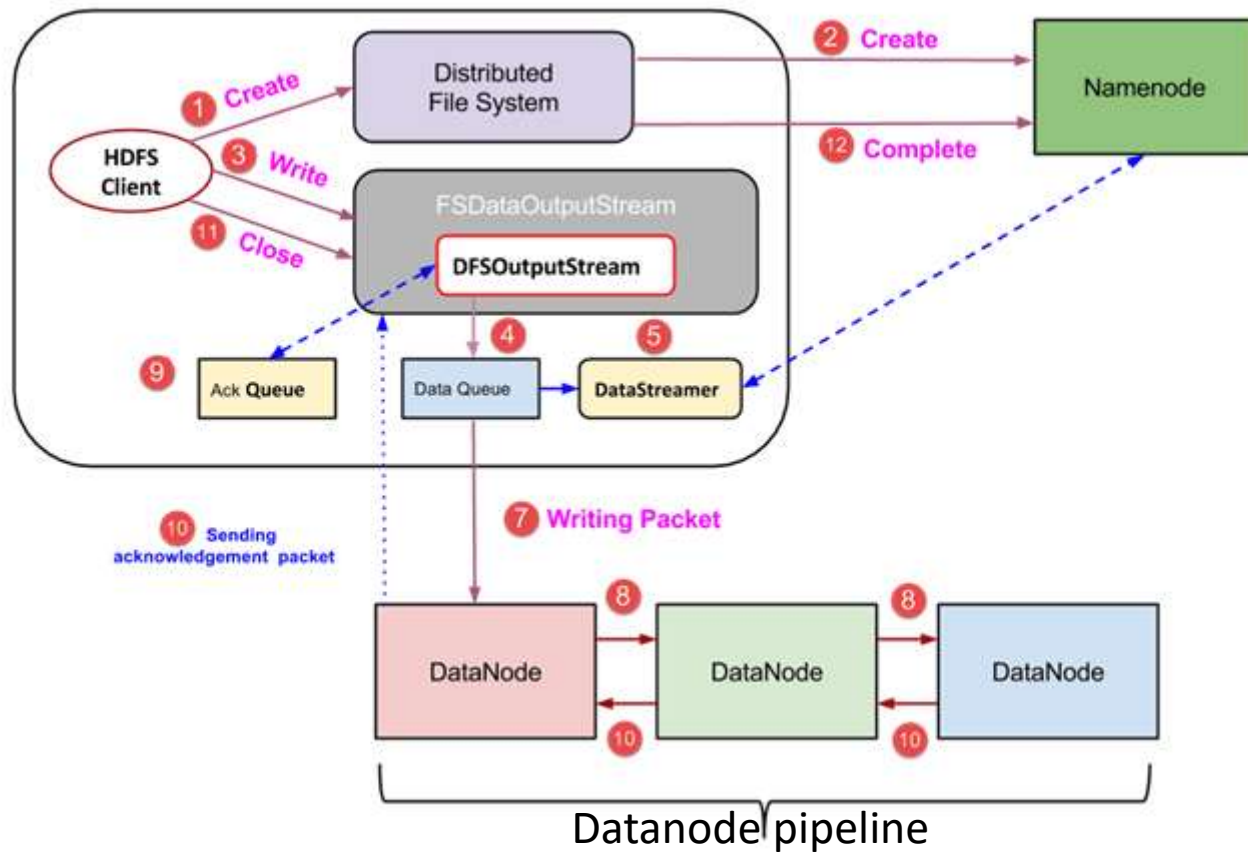
- HDFS takes a simple approach:
 - See the network as a tree
 - **Distance between two nodes is the sum of their distances to their closest common ancestor**

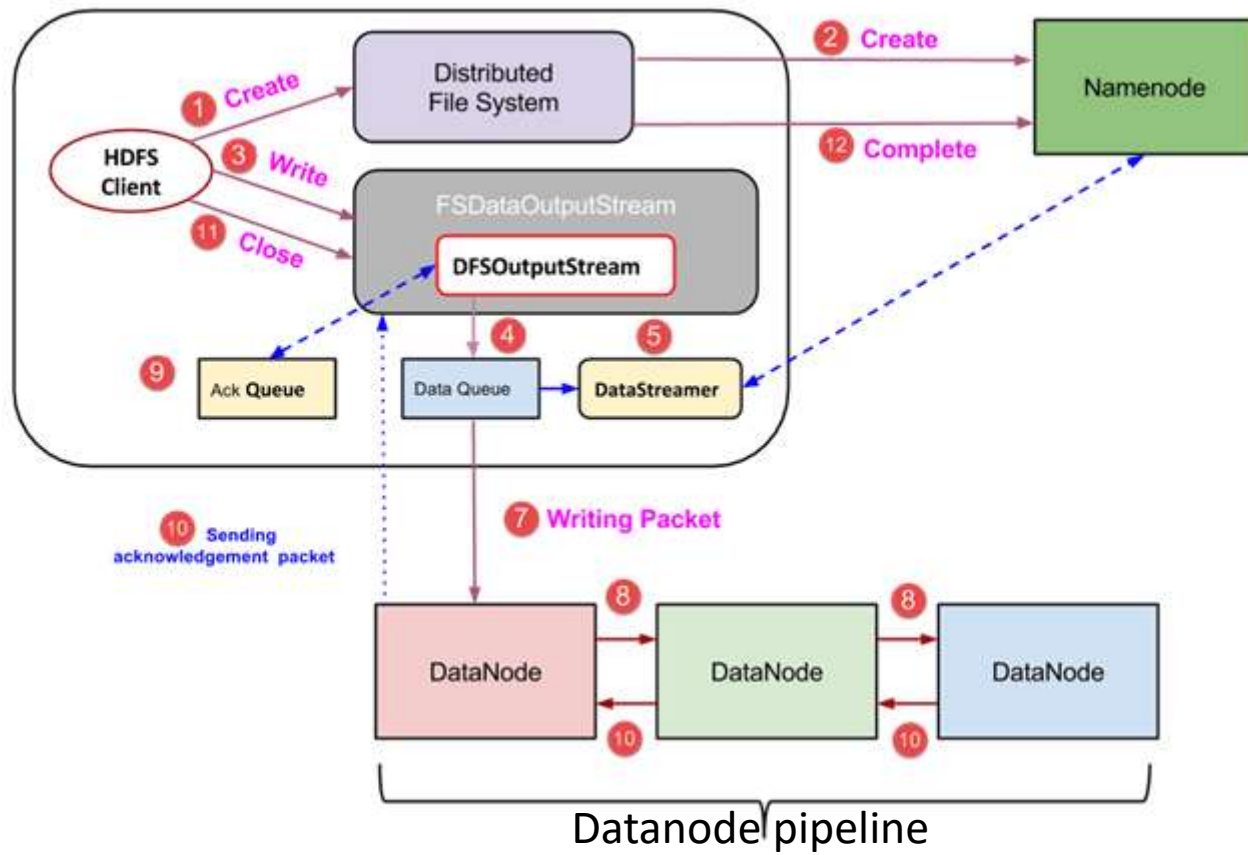


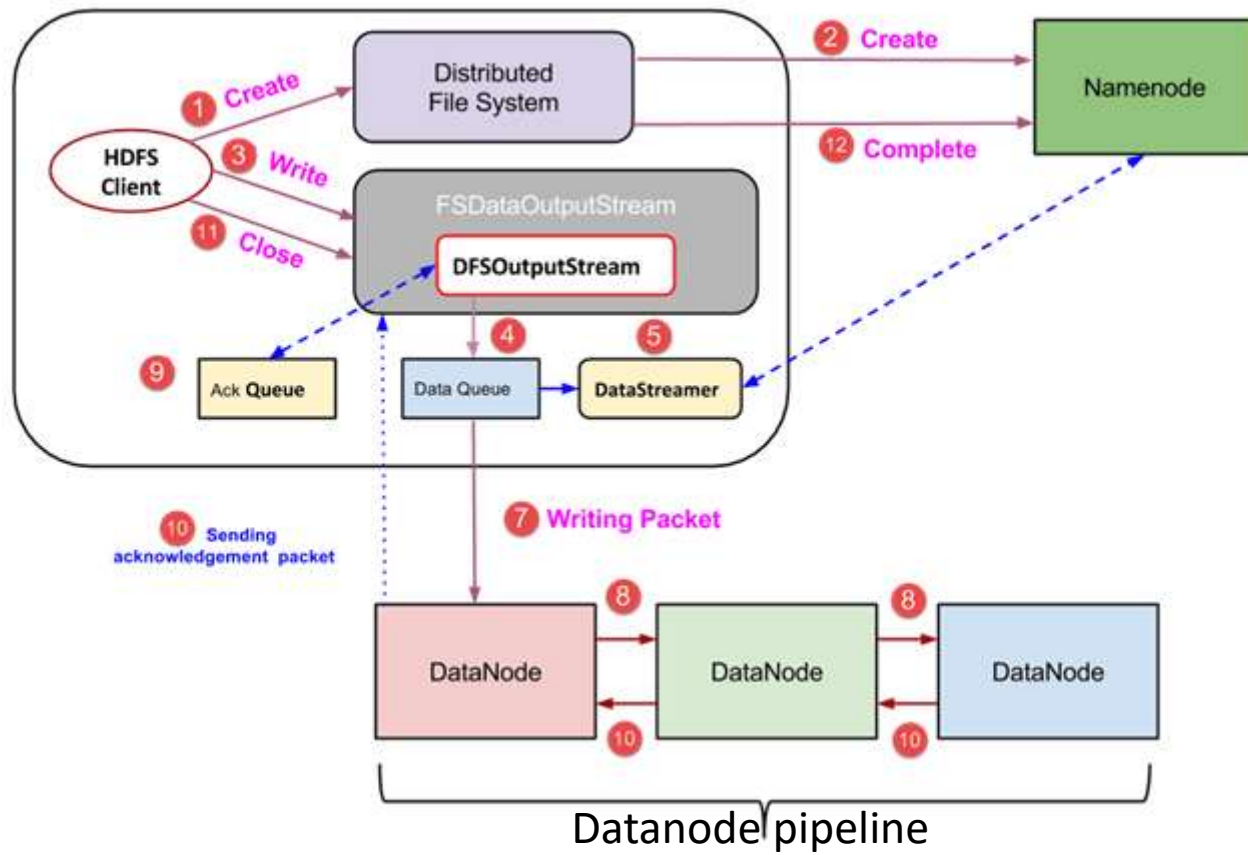
HDFS Inside: Write

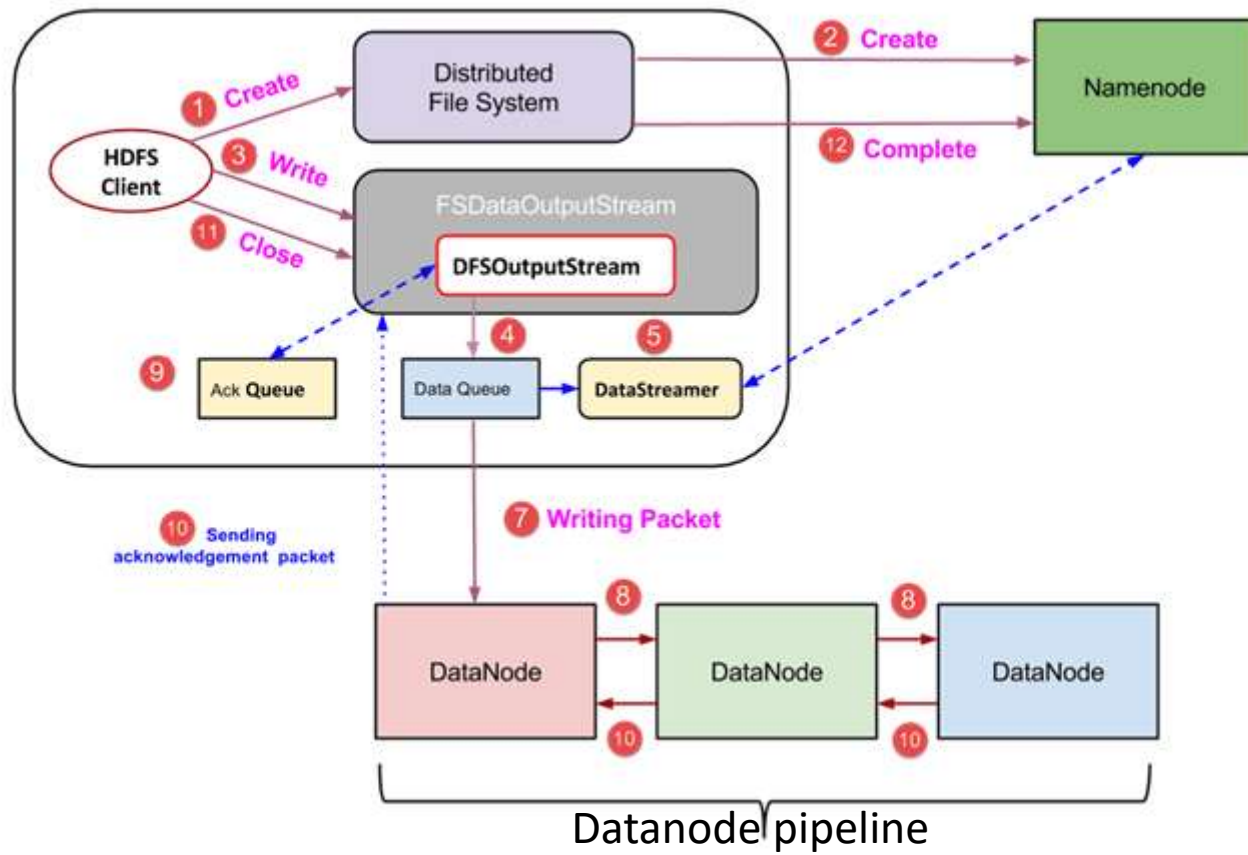


1. Client connects to NN to write data
2. NN tells client write these data nodes
3. Client writes blocks directly to data nodes with desired replication factor
4. In case of node failures, NN will figure it out and replicate the missing blocks









HDFS Inside: Write

- Q: Where should HDFS put the three replicas of a block? What tradeoffs we need to consider?
- Tradeoffs:
 - Reliability
 - Write Bandwidth
 - Read Bandwidth

Q: What are some possible strategies?

HDFS Inside: Write

- Replication Strategy vs Tradeoffs

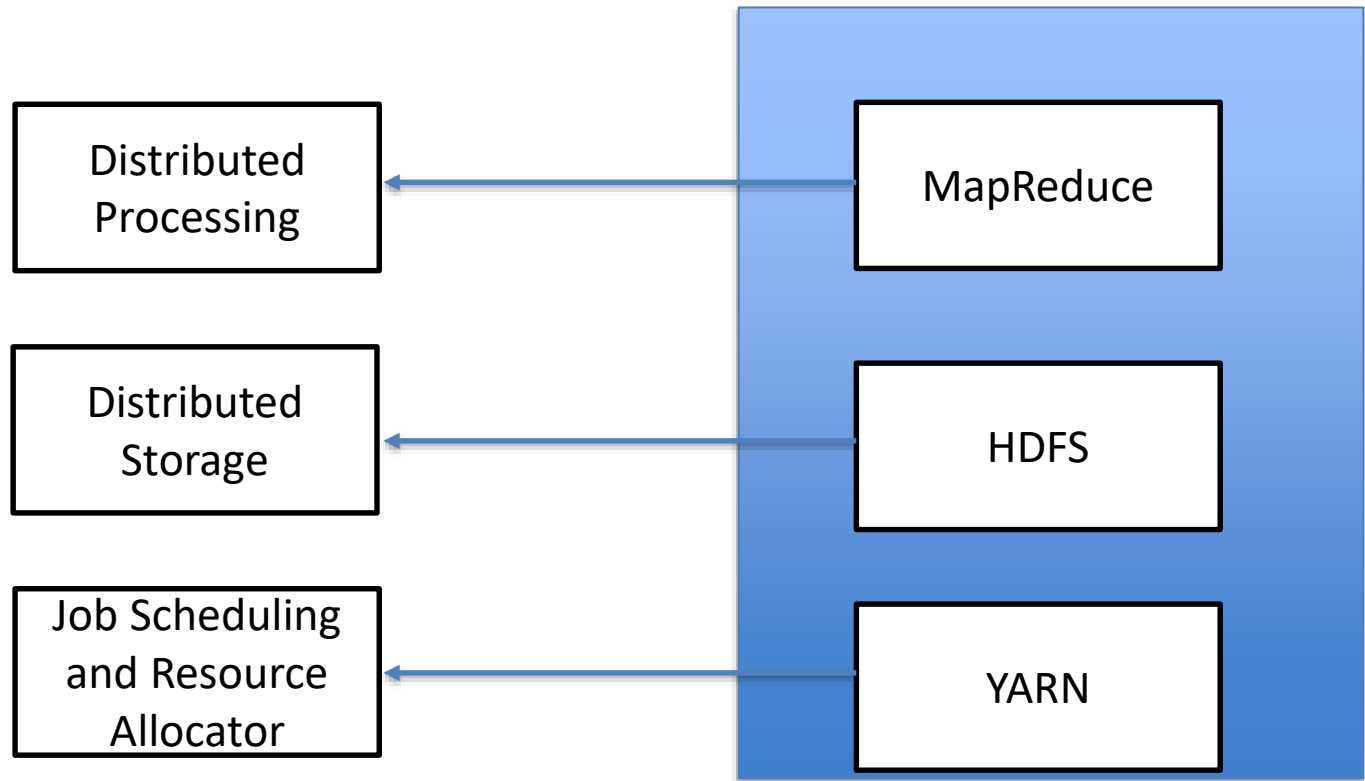
	Reliability	Write Bandwidth	Read Bandwidth
Put all replicas on one node			
Put all replicas on different racks			

HDFS Inside: Write

- Replication Strategy vs Tradeoffs

	Reliability	Write Bandwidth	Read Bandwidth
Put all replicas on one node			
Put all replicas on different racks			
HDFS: 1-> same node as client 2-> a node on different rack 3-> a different node on the same rack as 2			

MapReduce



MapReduce: A Real World Analogy

Coins Deposit



MapReduce: A Real World Analogy

Coins Deposit



Coins Counting Machine

MapReduce: A Real World Analogy

Coins Deposit



Mapper: Categorize coins by their face values

Reducer: Count the coins in each face value *in parallel*

MapReduce

- MapReduce is a programming model used for efficient processing in parallel over large data-sets in a distributed manner.

HDFS and MapReduce

- **Hadoop Distributed File System (HDFS):**
- **Storage System:** HDFS is a distributed file system designed to store and manage large volumes of data across multiple machines.
- **Data Replication:** HDFS divides large files into smaller blocks (typically 128 MB or 256 MB) and replicates these blocks across multiple nodes in a cluster to ensure fault tolerance.
- **Data Accessibility:** It provides a reliable and fault-tolerant storage layer, allowing data to be stored persistently and accessed by various applications running on a Hadoop cluster.

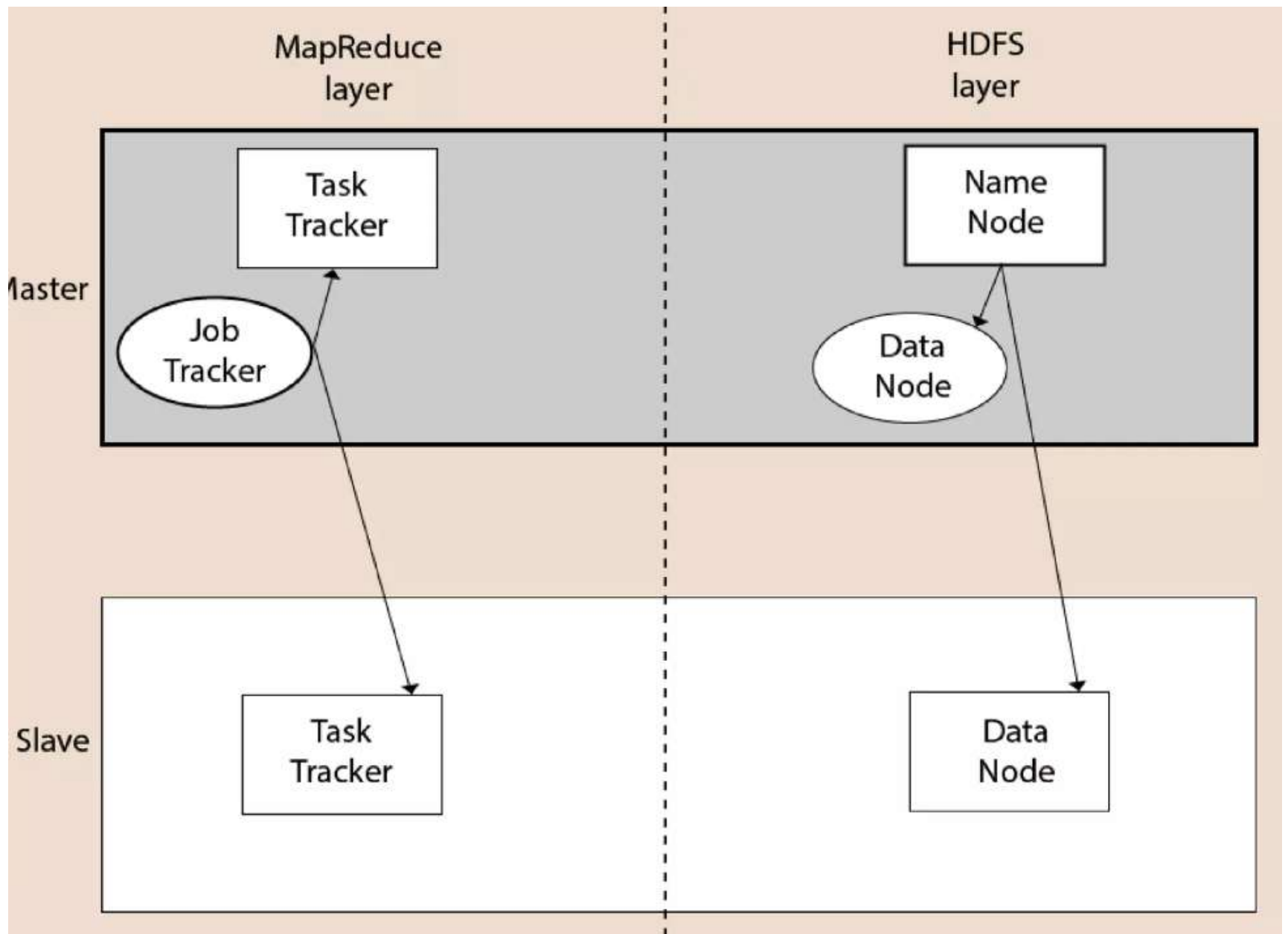
- **MapReduce:**
- **Processing Framework:** MapReduce is a programming model and processing engine used for parallel processing of large datasets.
- **Parallel Execution:** It processes data in parallel across a distributed cluster of computers, breaking tasks into two main phases: the "Map" phase for parallel processing, and the "Reduce" phase for aggregating results.
- **Scalability:** MapReduce enables scalable and efficient processing of vast amounts of data by distributing computation across multiple nodes in a Hadoop cluster.
- **Programming Model:** Developers write MapReduce programs using the Map and Reduce functions, allowing for easy parallelization of tasks.

Differences

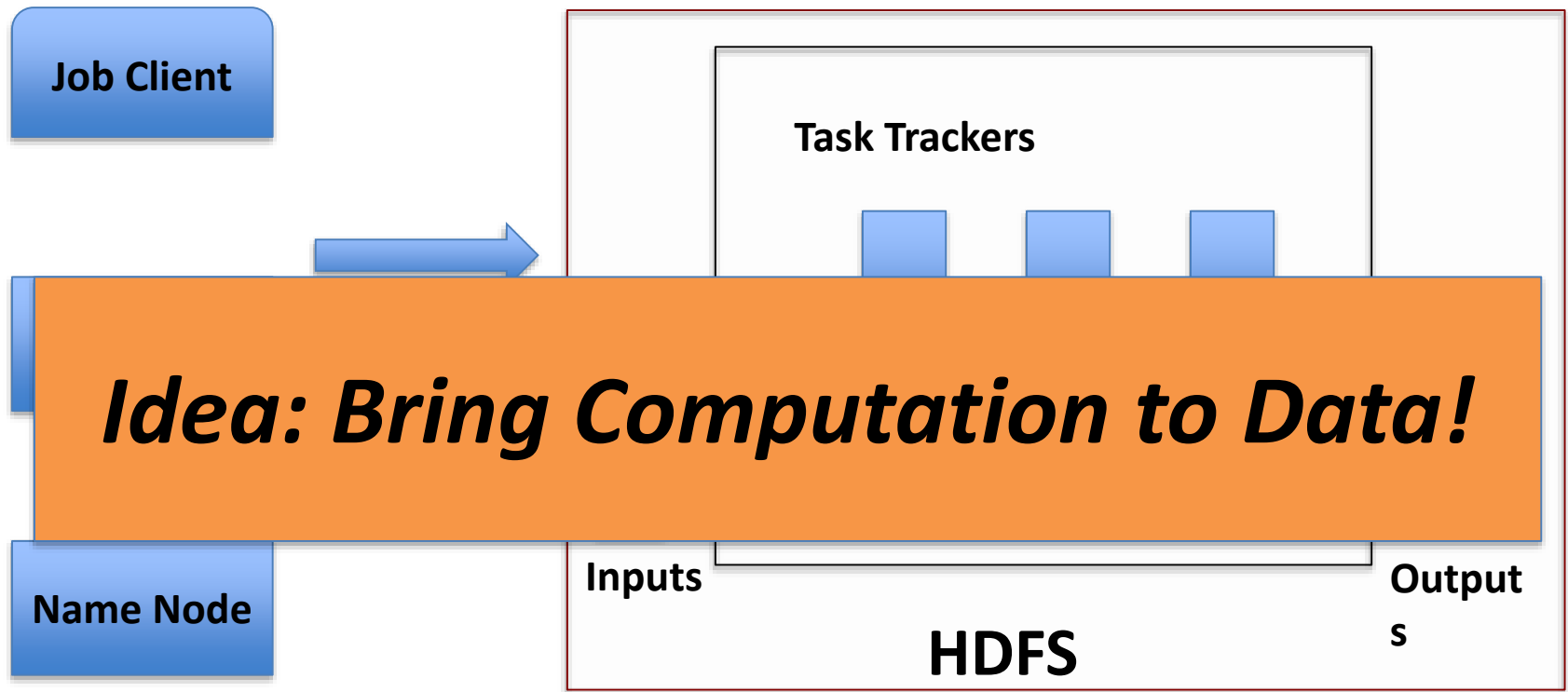
- **Purpose:** HDFS is primarily a distributed file storage system, while MapReduce is a data processing paradigm.
- **Functionality:** HDFS is responsible for storing and managing data, providing fault tolerance and scalability. MapReduce is focused on processing and analyzing the stored data in a parallel and distributed manner.
- **Components:** HDFS is the storage layer, while MapReduce is the processing layer built on top of HDFS. They often work together within the Hadoop ecosystem.
- **Use Cases:** HDFS is used for storing large datasets, and MapReduce is used for processing and analyzing those datasets.

Analogy: The Library and Librarians

- **HDFS (Library):**
 - Imagine a vast library (HDFS) with numerous shelves, each representing a node in a distributed system. Each shelf contains books, which are the data blocks stored in HDFS.
 - Librarians carefully organize and replicate these books across different shelves to ensure that even if one shelf (node) fails, the information is still available on other shelves. This reflects the fault-tolerant and distributed nature of HDFS.
- **MapReduce (Librarians):**
 - Now, think of the librarians (MapReduce) as data processors. Their job is to analyze and summarize the information in the books (data blocks) stored on the library shelves (HDFS).
 - Each librarian (MapReduce task) takes a specific section of books, reads and processes them independently (Map phase), and then collaborates with other librarians to consolidate the results (Reduce phase). This represents the parallel processing capabilities of MapReduce.
- **Working Together:**
 - The library (HDFS) provides a reliable and organized space to store a vast collection of information.
 - The librarians (MapReduce) take advantage of the organized collection, working simultaneously on different sections to efficiently process and analyze the wealth of knowledge stored in the library.



MapReduce Architecture: Master-Slaves

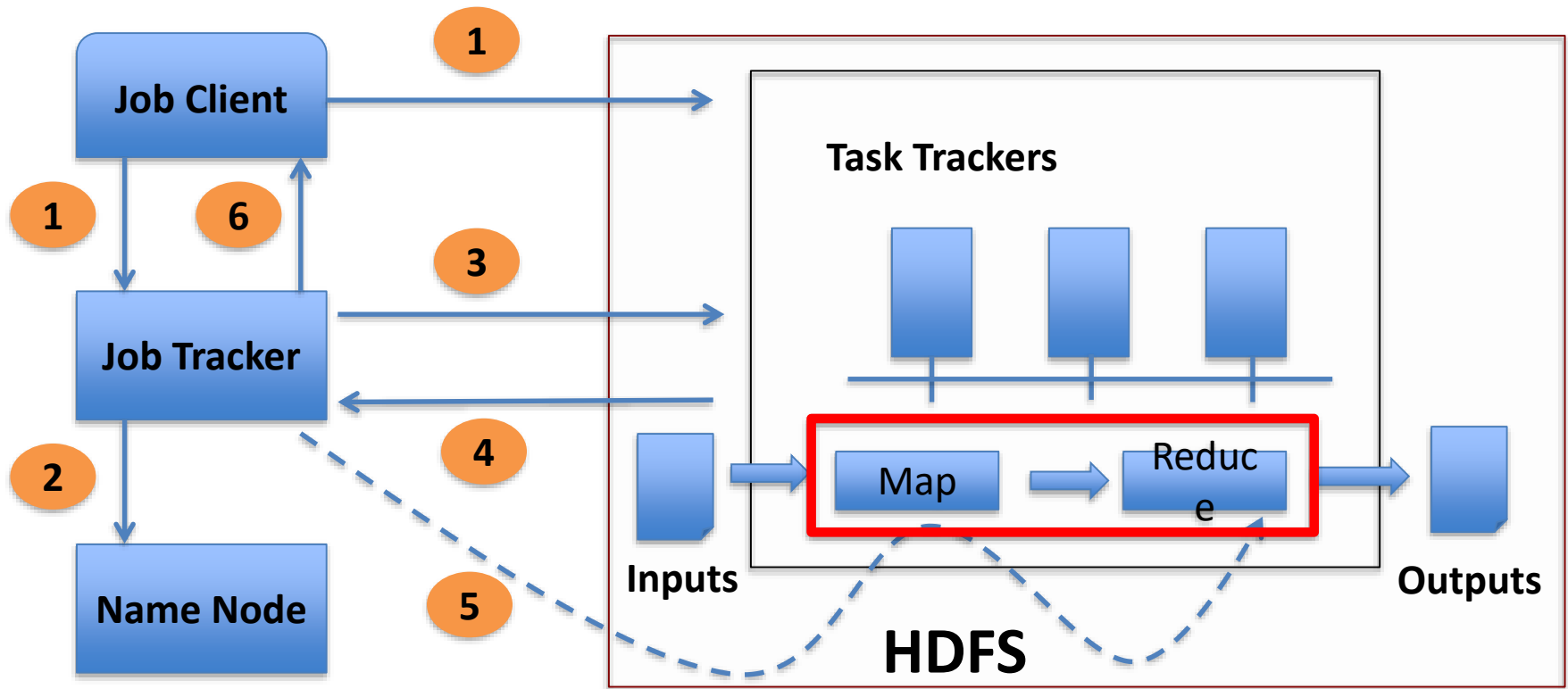


Job Client: Submit Jobs

Task Tracker: Execute Jobs

Job Tracker: Coordinate Jobs
(Scheduling, Phase Coordination, etc.)

MapReduce Architecture: Workflow



1. Client submits job to Job Tracker
2. Job Tracker talks to NN to find data it needs
3. Job Tracker creates execution plan and submits work to Task Trackers

4. Task trackers do the job and report progress/status to Job Tracker
5. Job Tracker manages task phases
6. Job Tracker finishes the job and updates status

Map-Reduce Programming Model

- Distributed File System
- Map functions operate to the data locally
- When you reduce these data chunks you're likely looking into some sort of commonality or pattern

Example Problem: Counting Words

- We have a huge text document and count the number of times each distinct word appears in the file
- **Sample application**
 - Analyze web server logs to find popular URLs
- How you solve this using a single machine?

Word Count

- **Case 1:** File too large for memory, but all <word, count> pairs fit in memory
 - You can create a big string array OR you can create a hash table
- **Case 2:** All <word, count> pairs do not fit in memory, but fit into disk
 - A possible approach (write computer programs/functions for each step)

1. Break the text document into sequence of words

getWords(textFile)

2. Sort the words

sort

3. Count the frequencies in a single pass

count

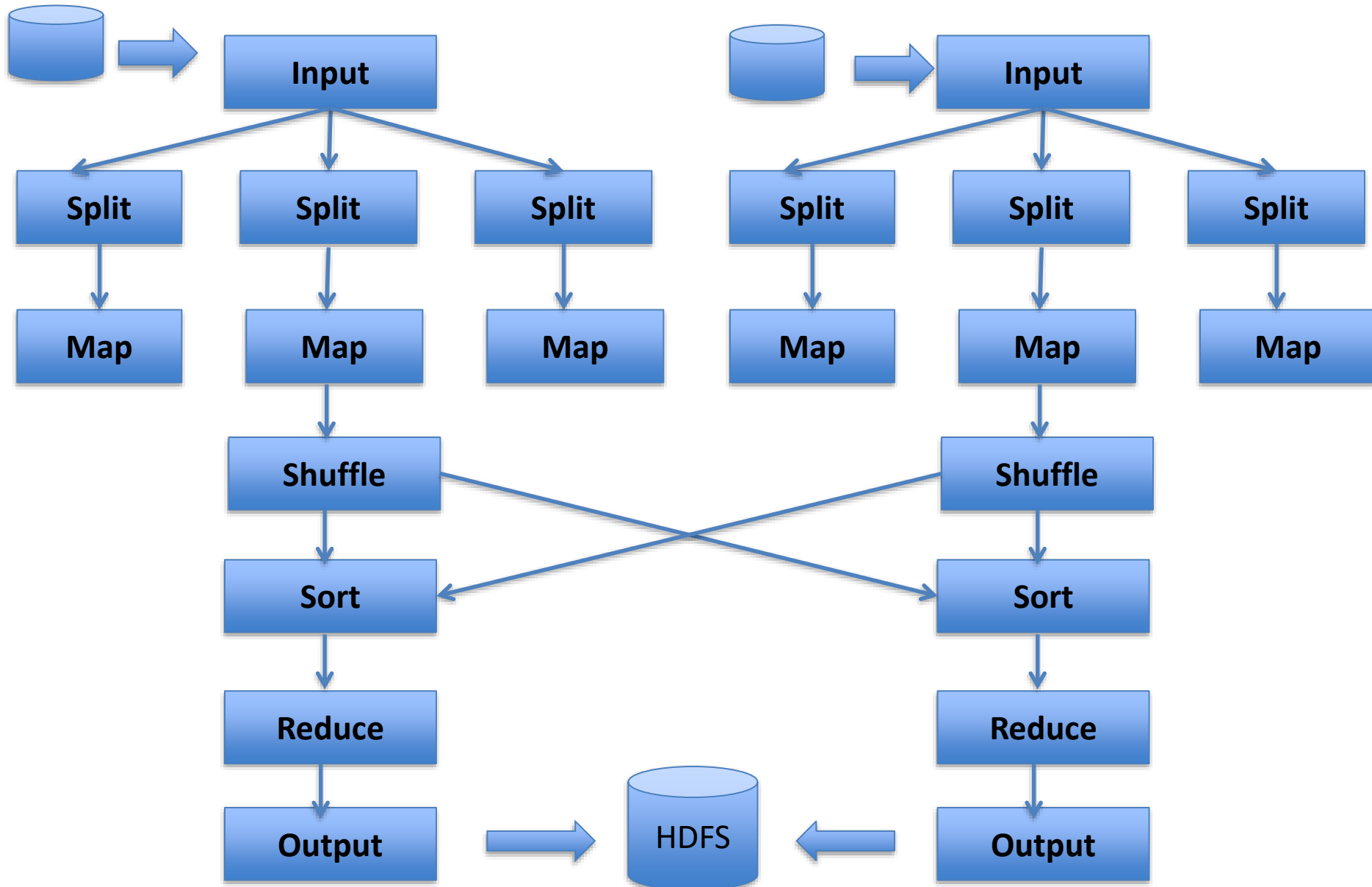
- This will bring same words together

- Case 2 captures the essence of Map-Reduce

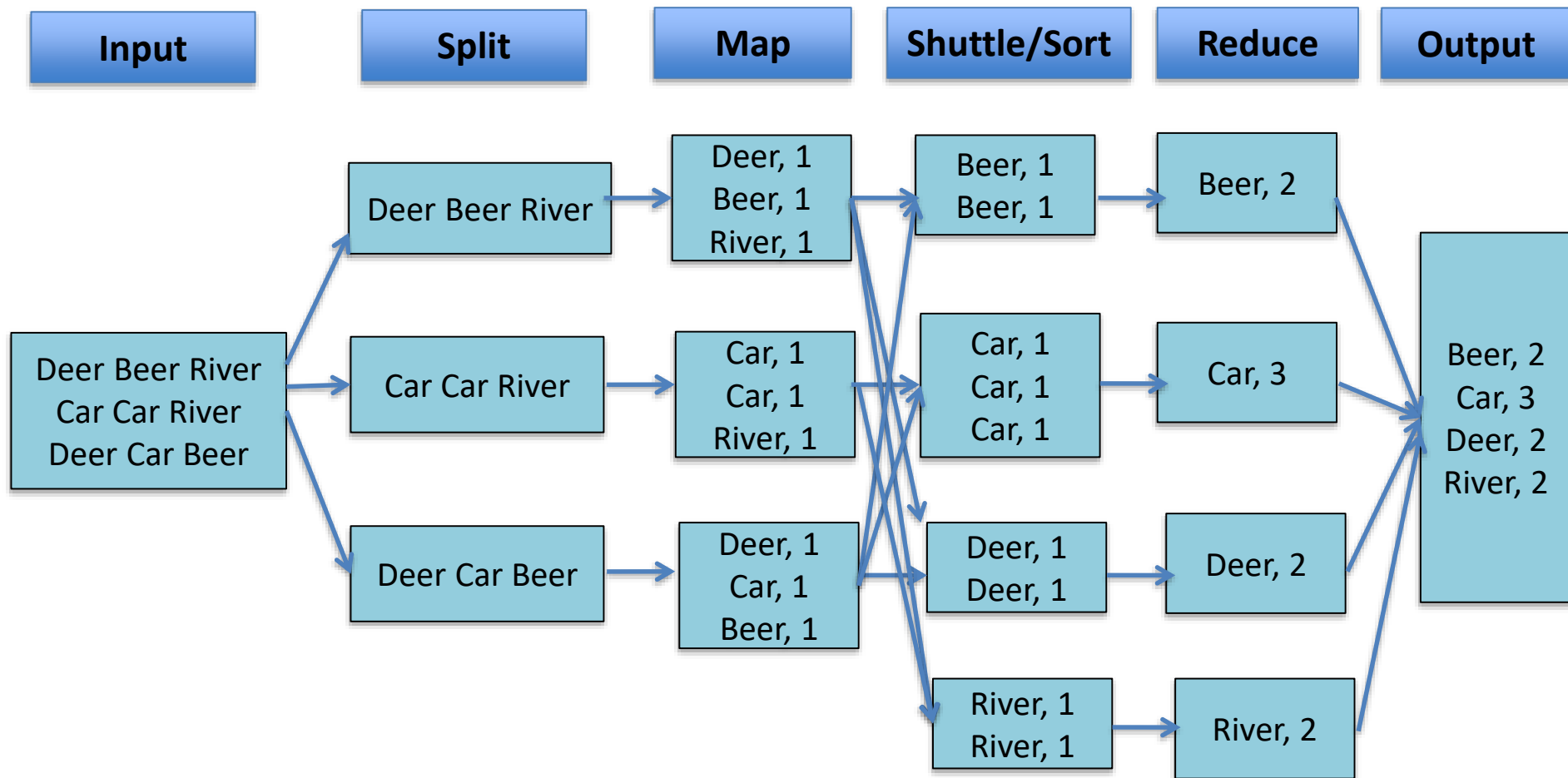
MapReduce Paradigm

- Implement two functions:
 - **Map** (k1,v1) -> list (k2, v2)
 - **Reduce**(k2, list(v2)) -> list (v3)
- Framework handles everything else
- Value with the **same key** go to the same reducer

MapReduce Internal

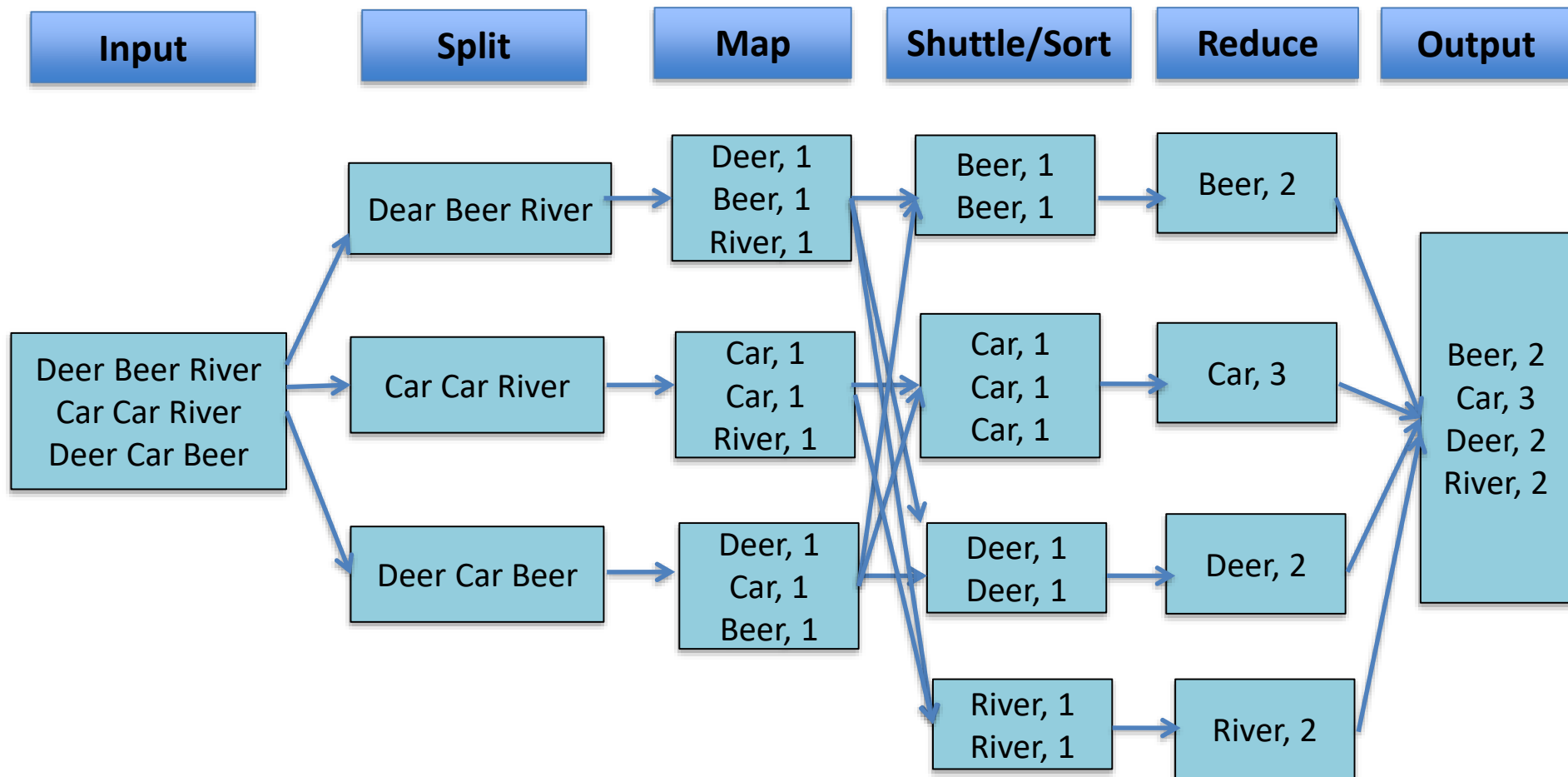


MapReduce Example: Word Count



Similar Flavor of Coins Deposit ? 😊

MapReduce Example: Word Count



Q: What are the Key and Value Pairs of Map and Reduce?

Map: Key=word, Value=1

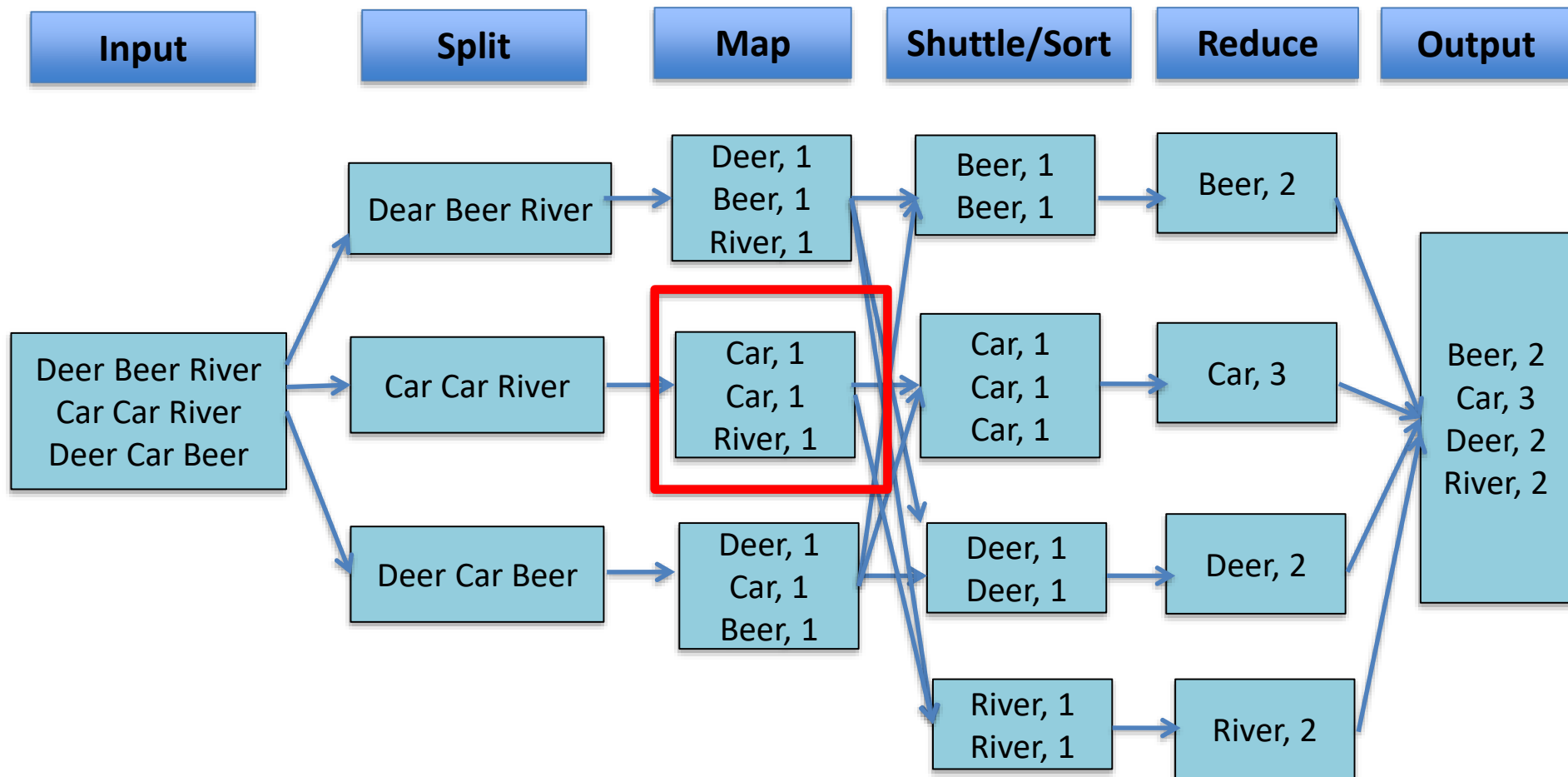
Reduce: Key=word, Value=aggregated count

Mapper and Reducer of Word Count

- Map(key, value){
 // key: line number
 // value: words in a line
 for each word w in value:
 Emit(w, "1");}
• Reduce(key, list of values){
 // key: a word
 // list of values: a list of counts
 int result = 0;
 for each v in values:
 result += ParseInt(v);
 Emit(key, result);}

Combiner is the same
as Reducer

MapReduce Example: Word Count

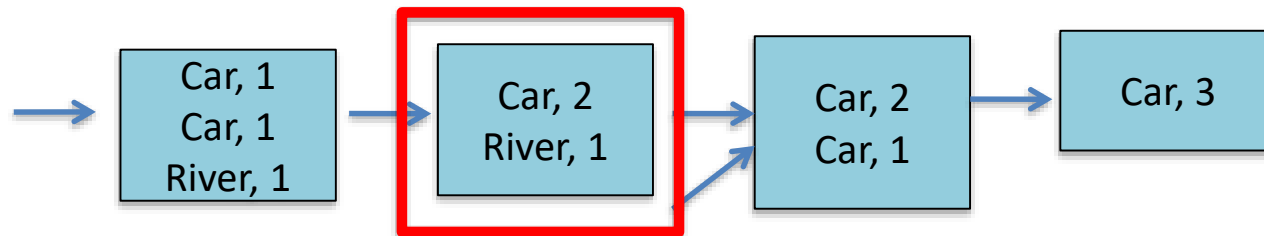


Q: Do you see any place we can improve the efficiency?

Local aggregation at mapper will be able to improve MapReduce efficiency.

MapReduce: Combiner

- **Combiner:** do local aggregation/combine task at mapper

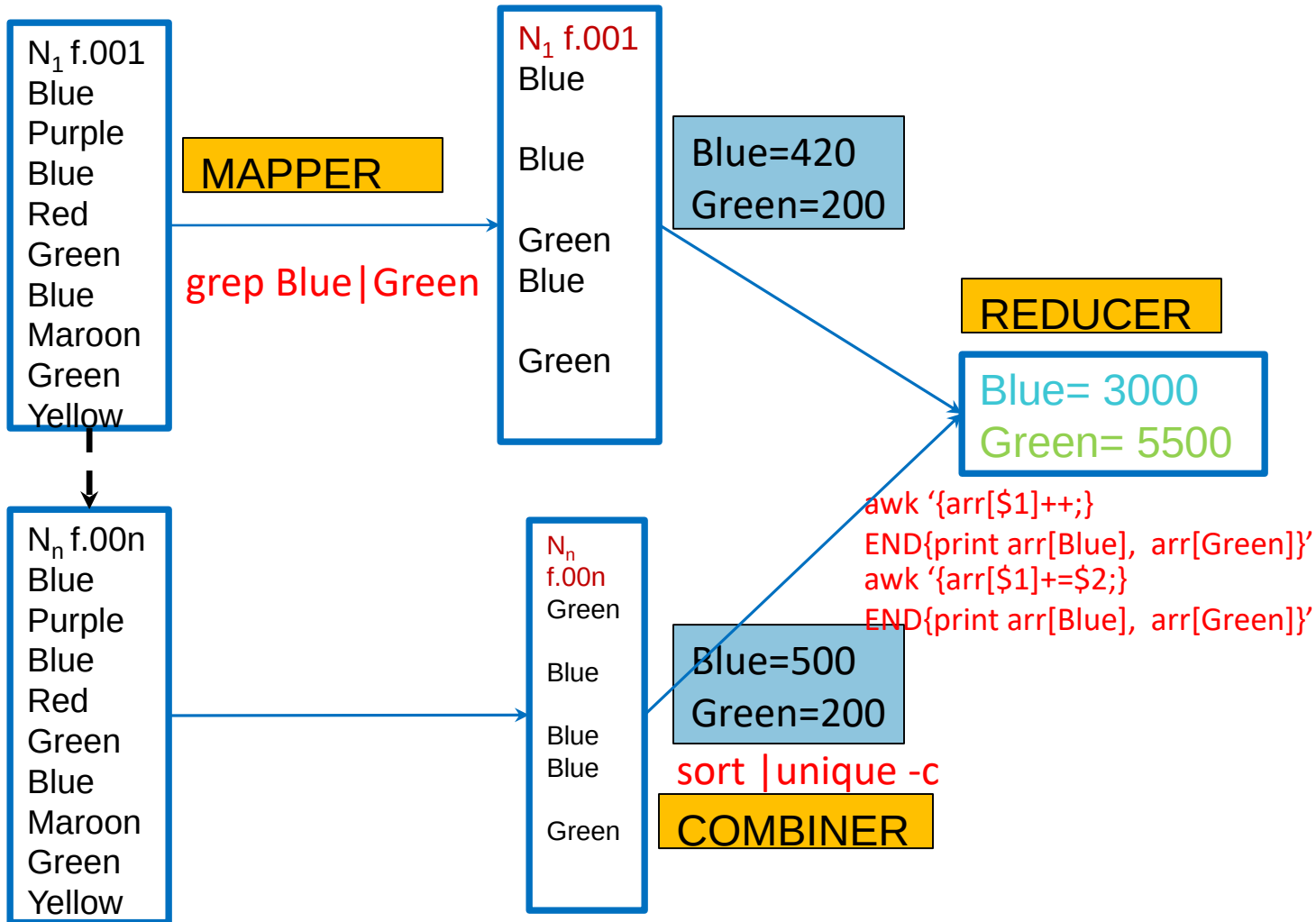


- **Q: What are the benefits of using combiner:**
 - Reduce memory/disk requirement of Map tasks
 - Reduce network traffic
- **Q: Can we remove the reduce function?**
 - No, reducer still needs to process records with same key but from *different mappers*
- **Q: How would you implement combiner?**
 - It is the same as Reducer!

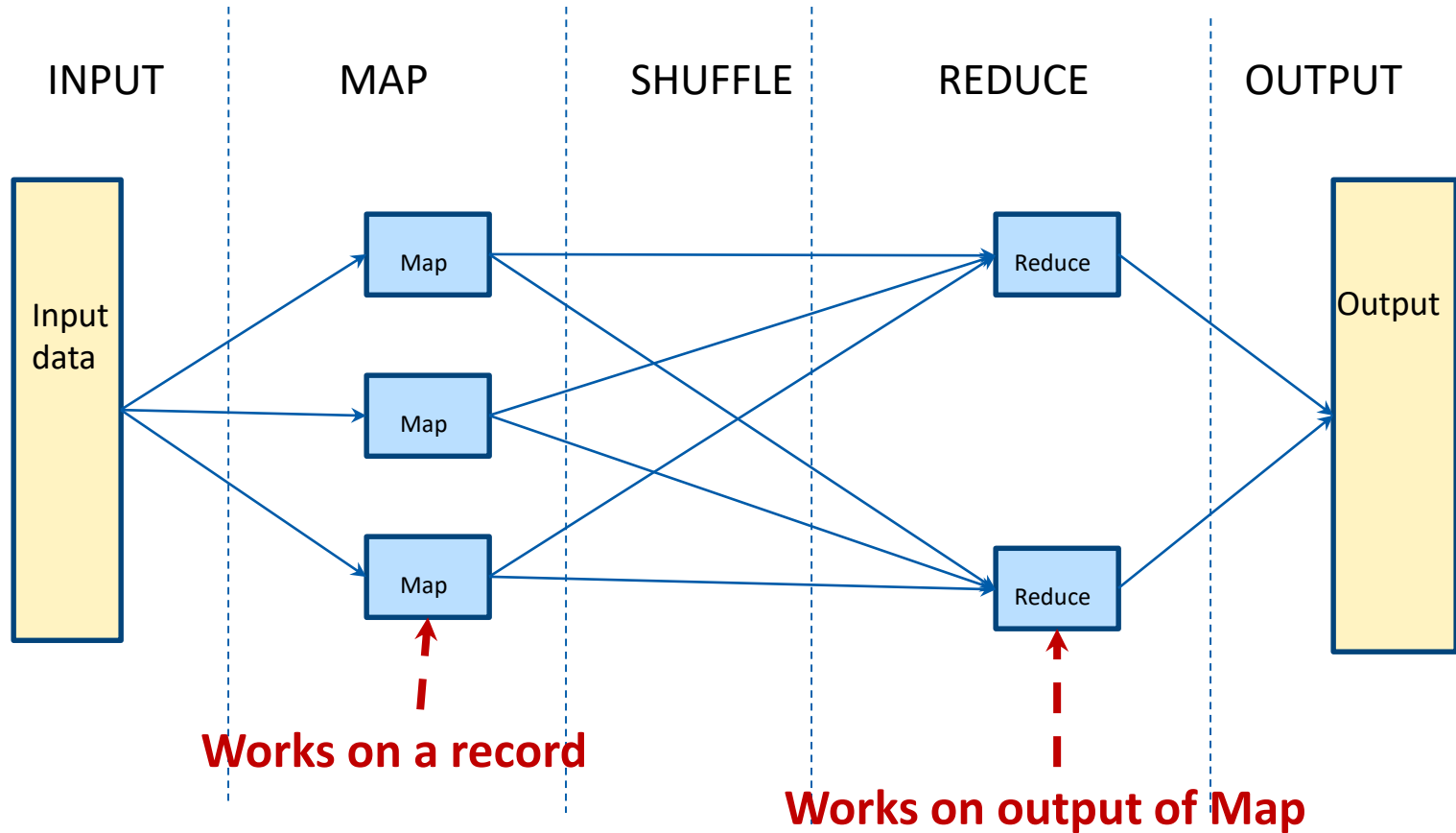
Example: Selecting Colors

Input: 1TB text file containing color names- Blue, Green, Yellow, Purple, Pink, Red, Maroon, Grey

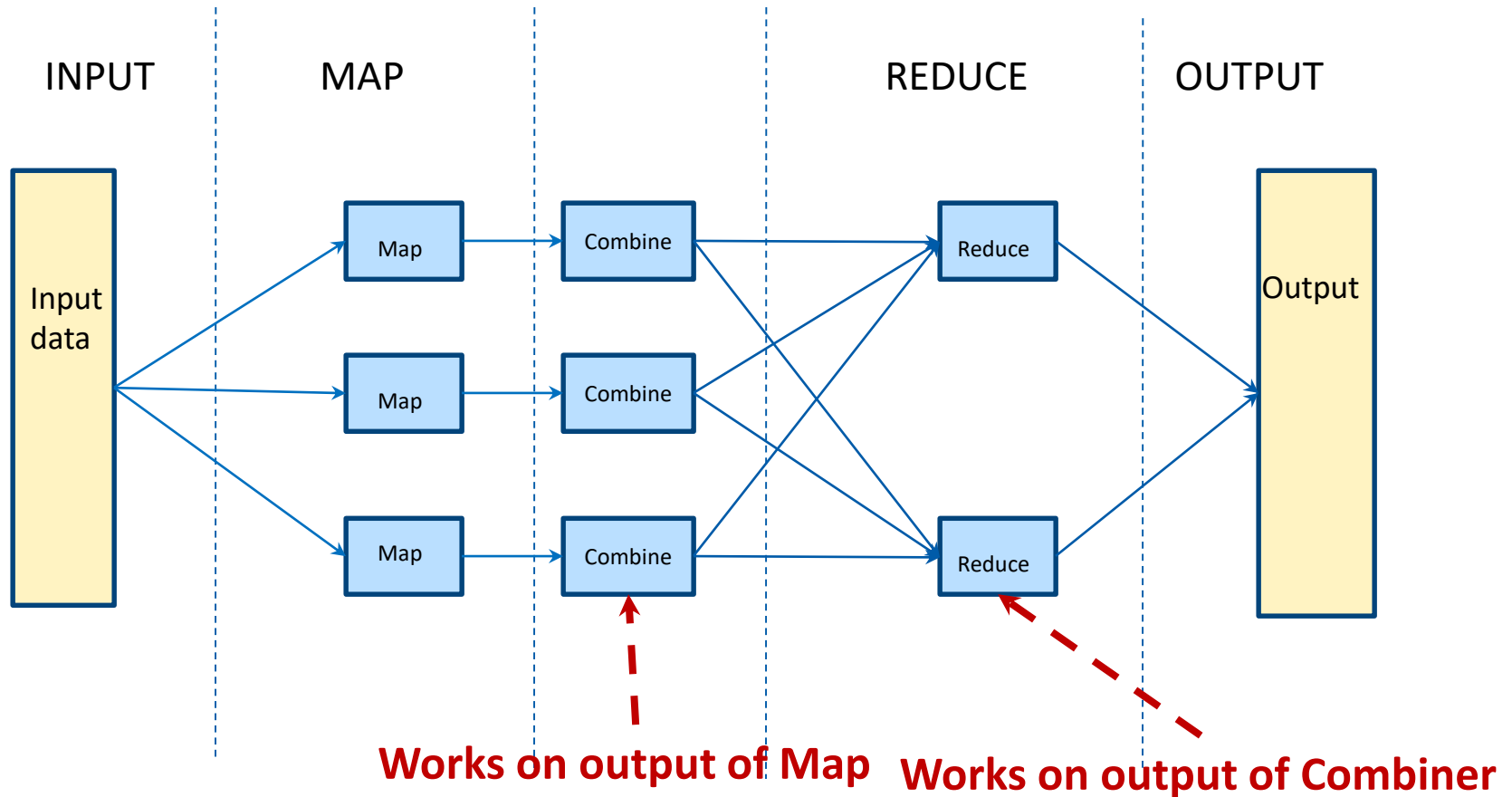
Desired output: Occurrence of colors Blue and Green



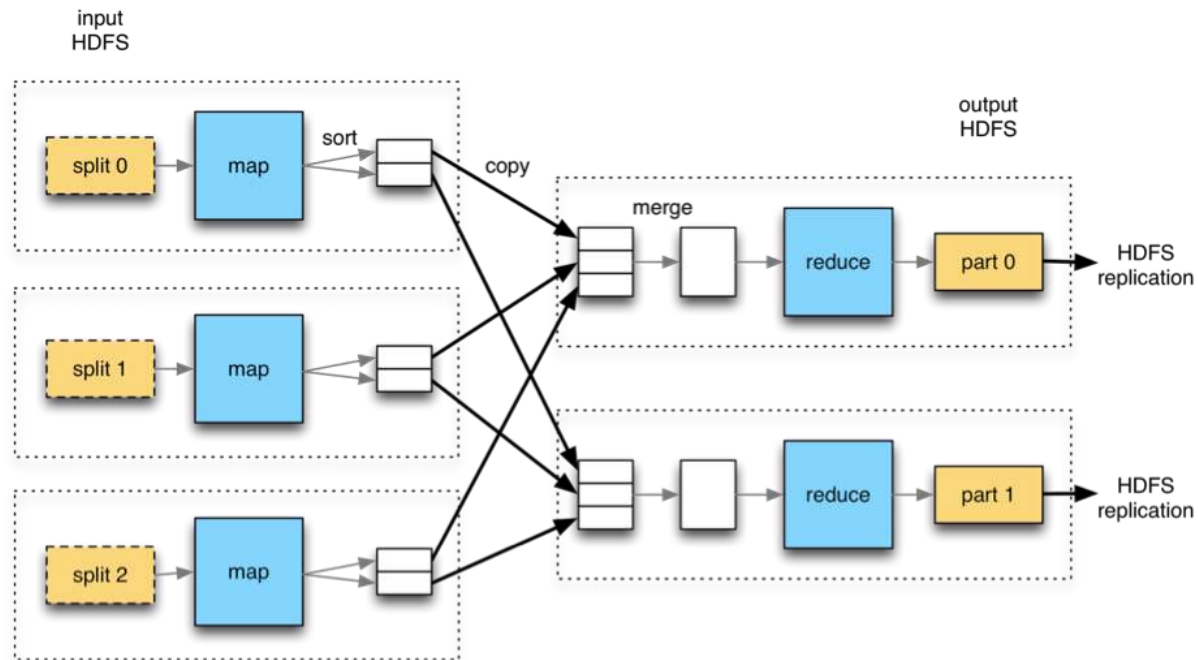
MapReduce Overview



MapReduce Overview



MapReduce Overview



Takes computation to data

MapReduce Summary

- ❑ Mapper, Reducer and Combiner act on <key, value> pairs
- ❑ Map function gets one record at a time as an input
- ❑ Combiner (if present) works on output of map
- ❑ Reducer works on output of map (or combiner, if present)
- ❑ Combiner can be thought of local-reducer
 - Reduces output of maps that are executed on same node

What Hadoop is not..

- ❑ Not for interactive file accessing
- ❑ Not meant for a large number of *small* files - but for a small number of *large* files
- ❑ MapReduce cannot be used for any and all applications

Your first program in hadoop (DEMO)

Open up any tutorial on hadoop and first program you see will be of wordcount 😊

Task: Given a text file, generate a list of words with the number of times each of them appear in the file

Input: Plain text file

Expected Output:

➤ *<word, frequency>* pairs for all words in the file

hadoop is a framework written in java
hadoop supports parallel processing
and is a simple framework

<hadoop, 2>	<framework , 2>	<supports , 1>
<is, 2>	<written , 1>	<parallel , 1>
<a , 2>	<in , 1>	<processing. , 1>
<java , 1>	<and,1>	<simple,1>

Mimicking the Hadoop Flow

- ❑ Create files “mapper.py” for Map and “reducer.py” for Reduce
- ❑ Mimic Hadoop using the Linux pipe (|)

```
cat input.txt | mapper.py | sort | reducer.py
```

hadoop is a framework written in java
hadoop supports parallel processing
and is a simple framework

```
cat input.txt | mapper.py | sort | reducer.py
```

```
a      2
and    1
framework  2
hadoop  2
in     1
is     2
java   1
parallel  1
processing  1
simple  1
supports 1
written 1
```


Your second program in hadoop

Task:

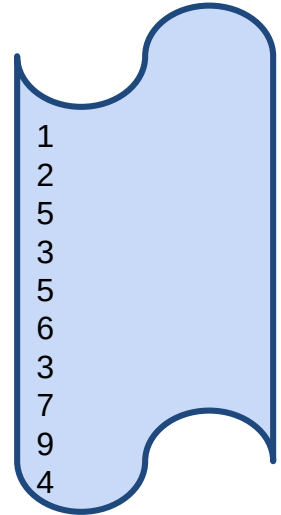
- Given a text file containing numbers, one per line, count sum of squares of odd, even and prime

Input:

- File containing integers, one per line

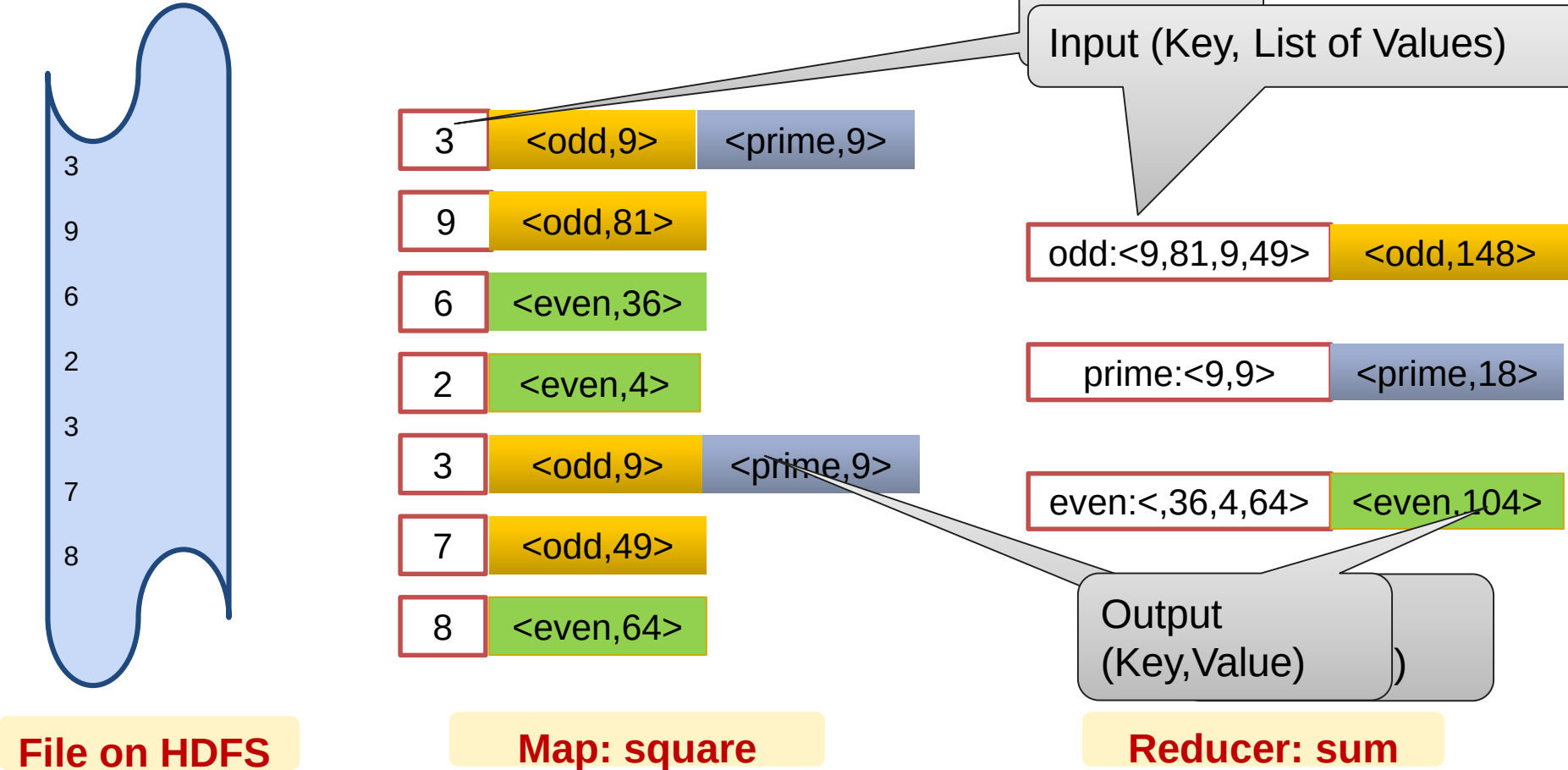
Expected Output:

- *<type, sum of squares>* for odd, even, prime



```
<odd, 302>  
<even, 278>  
<prime, 323 >
```

Your second program in hadoop



Your second program in hadoop : Exercises

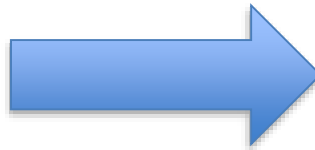
1. Mimic Hadoop Flow by writing appropriate mapper and reducer python scripts
2. Follow the tutorial to setup and run Single Node Hadoop cluster
3. Collaborate with others to setup and run Multi Node Hadoop cluster
 - Post on canvas any deviations from the steps given in the tutorial

Map Reduce Problems Discussion

- **Problem 1:** Find Word Length Distribution
- **Statement:** Given a set of documents, use Map-Reduce to find the length distribution of all words contained in the documents
- **Question:**
 - What are the Mapper and Reducer Functions?

This is a test data for
the word length
distribution problem

MapReduce



12: 1
7: 1
6: 1
4: 4
3: 2
2: 1
1: 1

Mapper and Reducer of Word Length Distribution

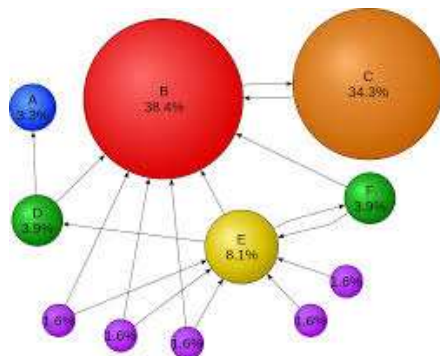
- `Map(key, value){`
 - `// key: document name`
 - `// value: words in a document`
 - `for each word w in value:`
 - `Emit(length(w), w);}`
- `Reduce(key, list of values){`
 - `// key: length of a word`
 - `// list of values: a list of words with the same length`
 - `Emit(key, size_of(values));}`

Map Reduce Problems Discussion

- **Problem 1:** Find Word Length Distribution
- **Mapper and Reducer:**
 - Mapper(document)
 { Emit (**Length(word), word**) }
 - Reducer(output of map)
 { Emit (Length(word), **Size of (List of words at a particular length)**) }

Map Reduce Problems Discussion

- **Problem 2:** Indexing & Page Rank
- **Statement:** Given a set of web pages, each page has a **page rank** associated with it, use Map-Reduce to find, for each word, a list of pages (sorted by rank) that contains that word
- **Question:**
 - What are the Mapper and Reducer Functions?



MapReduce



Word 1: [page x1,
page x2, ..]

Word 2: [page y1,
page y2, ...]

...

Mapper and Reducer of Indexing and PageRank

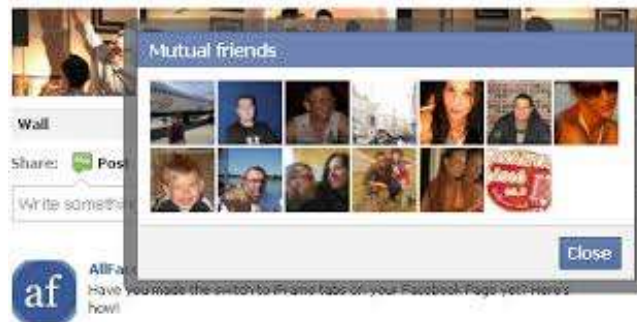
- `Map(key, value){`
 // key: a page
 // value: words in a page
 for each word `w` in value:
 `Emit(w, <page_id, page_rank>);`
- `Reduce(key, list of values){`
 // key: a word
 // list of values: a list of pages containing that word
 `sorted_pages=sort(values, page_rank)`
 `Emit(key, sorted_pages);`

Map Reduce Problems Discussion

- **Problem 2: Indexing and Page Rank**
- **Mapper and Reducer:**
 - Mapper(page_id, <page_text, page_rank>)
 { Emit (**word**, <**page_id**, **page_rank**>) }
 - Reducer(output of map)
 { Emit (word, **List of pages contains the word sorted by their page_ranks**) }

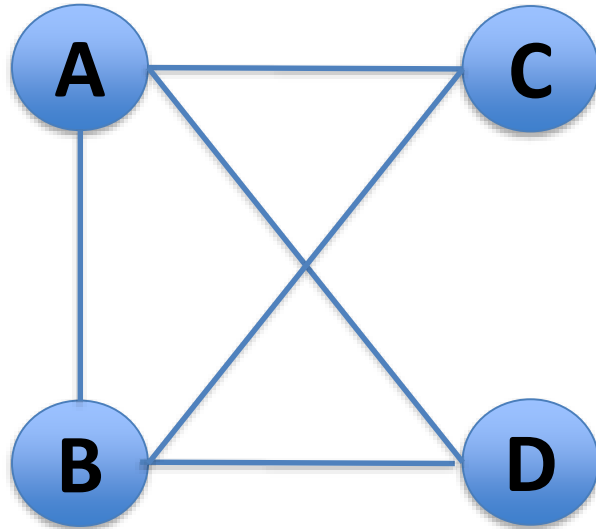
Map Reduce Problems Discussion

- **Problem 3:** Find Common Friends
- **Statement:** Given a group of people on online social media (e.g., Facebook), each has a list of friends, use Map-Reduce to find common friends of any two persons who are friends
- **Question:**
 - What are the Mapper and Reducer Functions?



Map Reduce Problems Discussion

- **Problem 3: Find Common Friends**
- **Simple example:**



Input:

A -> B,C,D

B -> A,C,D

C -> A,B

D -> A,B

Output:

(A ,B) -> C,D

(A,C) -> B

(A,D) -> ..

....

MapReduce

Mapper and Reducer of Common Friends

- `Map(key, value){`
 `// key: person_id`
 `// value: the list of friends of the person`
 `for each friend f_id in value:`
 `Emit(<person_id, f_id>, value);}`
- `Reduce(key, list of values){`
 `// key: <friend pair>`
 `// list of values: a set of friend lists related with the friend`
 `pair`
 `for v1, v2 in values:`
 `common_friends = v1 intersects v2;`
 `Emit(key, common_friends);}`

Map Reduce Problems Discussion

- **Problem 3: Find Common Friends**
- **Mapper and Reducer:**
 - Mapper(friend list of a person)
 - { for each person in the friend list:
Emit (<friend pair>, <list of friends>) }
 - Reducer(output of map)
 - { Emit (<friend pair>, Intersection of two
(i.e, the one in friend pair) friend lists)}

Map Reduce Problems Discussion

- **Problem 3: Find Common Friends**
- **Mapper and Reducer:**

Input:

A -> B,C,D
B-> A,C,D
C-> A,B
D->A,B

Map:

(A,B) -> B,C,D

(A,C) -> B,C,D

(A,D) -> B,C,D

(A,B) -> A,C,D

(B,C) -> A,C,D

(B,D) -> A,C,D

(A,C) -> A,B

(B,C) -> A,B

(A,D) -> A,B

(B,D) -> A,B

Reduce:

(A,B) -> C,D

(A,C) -> B

(A,D) -> B

(B,C) -> A

(B,D) -> A

*Suggest
Fiends 😊*

References

- ❑ Official Hadoop website- <http://hadoop.apache.org/>
- ❑ Hadoop presentation wiki-
<http://wiki.apache.org/hadoop/HadoopPresentations?action=AttachFile>
- ❑ <http://developer.yahoo.com/hadoop/>
- ❑ <http://wiki.apache.org/hadoop/>
- ❑ <http://www.cloudera.com/hadoop-training/>
- ❑ <http://developer.yahoo.com/hadoop/tutorial/module2.html#basics>

Actual Hadoop Flow

- ❑ <http://www.michael-noll.com/tutorials/writing-an-hadoop-mapreduce-program-in-python/>
- Installation (From the above page)
 - [Running Hadoop On Ubuntu Linux \(Single-Node Cluster\)](#) – How to set up a *pseudo-distributed, single-node* Hadoop cluster backed by the Hadoop Distributed File System (HDFS)
 - [Running Hadoop On Ubuntu Linux \(Multi-Node Cluster\)](#) – How to set up a *distributed, multi-node* Hadoop cluster backed by the Hadoop Distributed File System (HDFS)
- Minor changes needed due to changes in recent hadoop distribution directory

Actual Hadoop Flow

: Snippets from <http://www.michael-noll.com/tutorials/writing-an-hadoop-mapreduce-program-in-python/>

❑ Copy input to HDFS

```
$ bin/hadoop dfs -copyFromLocal /tmp/gutenberg /user/hduser/gutenberg
```

❑ Run the mapper and reducer

```
$ bin/hadoop jar <path-to-jar>/hadoop-*streaming*.jar \  
  -file /home/hduser/mapper.py -mapper /home/hduser/mapper.py-file \  
  /home/hduser/reducer.py -reducer /home/hduser/reducer.py \  
  -input /user/hduser/gutenberg/* -output /user/hduser/gutenberg-output
```

Actual Hadoop Flow

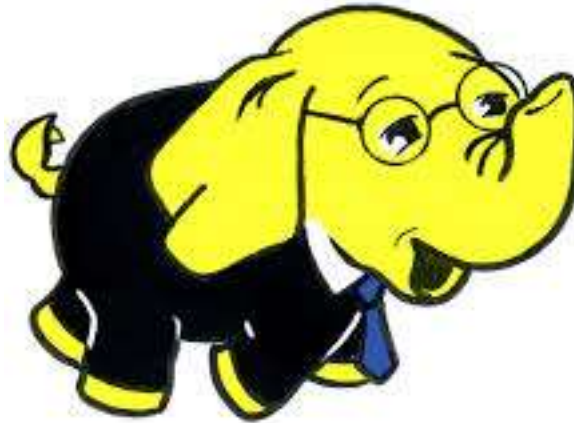
: Snippets from <http://www.michael-noll.com/tutorials/writing-an-hadoop-mapreduce-program-in-python/>

❑ Check the output

```
$ bin/hadoop dfs -cat /user/hduser/gutenberg-output/part-00000
"(Lo)cra"      1
"1490      1
"1498," 1
"35" 1
"40," 1
"A 2
"AS-IS".      2
"A_ 1
"Absoluti     1
[...]
hduser@ubuntu:/usr/local/hadoop$
```

Further Reading

- ❑ [Hadoop: The Definitive Guide: Tom White](#)
- ❑ [http://developer.yahoo.com/hadoop/tutorial/](#)
- ❑ [http://www.cloudera.com/content/cloudera-content/cloudera-docs/HadoopTutorial/CDH4/Hadoop-Tutorial.html](#)



Enjoy MR and Hadoop 😊
Thank you